CALIBER: A Layered Control Plane for Governed Releases of AI-Agent Resources

Reza Rahimi
reza.rahimi@jazzx.ai

Technical Report
Contextual Adaptive Lifecycle for Intelligent Build, Evaluation, and Refinement

Abstract

Production LLM-agent systems are well observed, while release governance remains uneven across the resources they depend on. Tracing tells an operator that a prompt regressed; the toolchain around it says little about what to change, whether the change is better, or how to undo it. Prompt-lifecycle platforms now close part of this for prompts — versions, a named pointer resolved at call time, rollback by re-pointing — but not for the workflows, tool definitions, MCP servers, and retrieval corpora an agent equally depends on, and not with the evaluation result acting as a precondition rather than an annotation.

We present CALIBER, a layered control plane for the lifecycle of AI-agent resources. Its organizing abstraction is the *governed asset*: a typed record carrying version history, an authority model, an evidence base, and — where its family supports one — a live target that client code resolves at call time. CALIBER spans nine asset families in 170k lines of Python and 202k of TypeScript, reusing MLflow rather than rebuilding it.

Layering makes shared capabilities available; explicit per-family wiring determines the guarantees. We report that surface in full. Prompt authoring is non-live, and every routed prompt-alias mutation uses a durable intent, idempotent provider operation, and observable reconciliation state; equivalent crash handling is not yet uniform across other families. The quantitative evaluation is specified but not executed; every unpopulated result is marked rather than estimated.

Table of Contents

1 Introduction . . . 5
2 Motivation . . . 8
2.1 A failure no single resource lifecycle closes . . . 8
2.2 Why the obvious answers do not work . . . 8
2.3 What a solution has to provide . . . 9
3 Background and Related Work . . . 10
3.1 ML lifecycle platforms . . . 10
3.2 Agent architectures and composition frameworks . . . 10
3.3 Prompt and program optimization . . . 10
3.4 Evaluation, judges, and human alignment . . . 11
3.5 Observability and prompt-lifecycle platforms . . . 11
3.6 Isolation, policy, and provenance . . . 11
3.7 Distributed-systems foundations . . . 12
4 System Overview . . . 13
4.1 Design principles . . . 13
4.2 The governed asset . . . 15
4.3 Six lifecycle modes . . . 16
4.4 The governance chain . . . 16
4.5 Reading the two architectural diagrams . . . 17
5 Multi-layer Architecture . . . 18
5.1 Availability, not inheritance . . . 18
5.2 L_1: infrastructure . . . 19
5.3 L_2: kernel services . . . 21
5.4 L_3: the governance substrate . . . 21
5.5 L_4: the asset families . . . 21
5.6 L_5 lifecycle modes and L_6 surfaces . . . 21
5.7 Control flow, data flow, and the execution lifecycle . . . 21
5.8 Extensibility . . . 27
5.9 Scalability, reliability, and fault tolerance . . . 29
6 Core Components . . . 31
6.1 The registries . . . 31
6.2 The evidence base . . . 31
6.3 The evaluation framework . . . 31
6.4 The calibration framework . . . 32
6.5 Gate semantics: the distinction the system turns on . . . 32
6.6 Release control . . . 34
6.7 The trust boundary . . . 36
6.8 Published services and the copilot . . . 36
7 Capabilities . . . 39
7.1 R2 in practice: late-bound live targets . . . 39
7.2 R3 in practice: evidence bound to versions . . . 39

7.3 R4 in practice: one enforced gate, one human in refinement 40
7.4 Automation reach, stated as a boundary 40
7.5 Cross-family capabilities 40
7.6 Capability coverage against comparable systems 41
8 Design Decisions 43
8.1 D1: reuse MLflow rather than owning the substrate 43
8.2 D4: a human at refinement APPLY, always 43
8.3 D5: database-arbitrated queues, no worker tier 45
8.4 D9: a single environment in v1 45
9 Implementation 46
9.1 Scale and composition 46
9.2 Three implementation choices worth recording 46
9.3 Testing and quality gates 47
9.4 Deployment 47
10 Evaluation 48
10.1 What state this evaluation is in 48
10.2 Evaluation questions and why these 48
10.3 Protocol summary 50
10.4 Threats to validity we can already name 50
11 Comparison with Existing Frameworks 52
11.1 The prompt-lifecycle platforms already do much of this 52
11.2 Why a ranking across the broader field would be the wrong output 52
11.3 The dimensions where CALIBER is genuinely different 55
11.4 The dimensions where CALIBER is behind 55
11.5 Positioning, stated plainly 56
12 Limitations 57
12.1 Limitations of this paper 57
12.2 Limitations of the release path 57
12.3 Limitations of the architecture 58
12.4 Limitations of the security model 58
12.5 Limitations of scope 59
13 Future Work 60
14 Conclusion 62
Availability 63
References 64
A Asset Families in Detail 69
B Interface Surface and the Governed-Asset Contract 70
B.1 The shape every registry presents 70
B.2 What a promote must record 70
B.3 Admission, in the order it happens 71
C Measurement Protocol 72
C.1 Common setup 72
C.2 E1 — Control-plane cost 72
C.3 E2 — Queue arbitration 72
C.4 E3 — The refinement path 73
C.5 E4 — Release and recovery 74
C.6 E6 — Baselines 74

C.7 E7 — Operators 75
C.8 Fault injection 75
C.9 Deviations to report 75
D Operational Configuration Notes 77

1 Introduction

An LLM-agent system in production is often thoroughly observable while governance varies sharply by resource family.

The observability half is well served. Tracing frameworks capture spans, tool calls, retrieved context, and token accounting; evaluation harnesses score outputs offline; LLM-as-judge methods approximate human preference at a fraction of the cost [39, 69]; and hosted platforms render all of it in a dashboard [7, 33, 35]. When an agent produces a bad answer, a competent team will find out, and will be able to point at the span where it happened.

What happens next is where the tooling stops. Someone opens a source file and edits a prompt string. Perhaps they test it on a handful of examples they assembled by hand; perhaps they compare against the previous version, or perhaps they only check that the specific reported failure is fixed. They merge, deploy, and move on. Six weeks later a different failure arrives, and nobody can reconstruct what the prompt used to be, what evidence justified the change, who approved it, or whether the change that fixed one class of input silently degraded three others. The remediation path is manual, undocumented, and irreproducible — precisely the asymmetry Sculley et al. described for conventional ML systems [60], displaced from model artifacts onto prompts, tool definitions, agent skills, and retrieval corpora.

This gap is not an oversight in any individual tool; it is a consequence of how the ecosystem is factored. Composition frameworks are built to answer *how do I build this* [32, 40]. Program optimizers are built to answer *what should this prompt say* [2, 29, 51]. Tracking and observability systems are built to answer *what happened* — and, increasingly, more than that: MLflow's Prompt Registry, LangSmith, and Langfuse now version prompts, expose a named pointer the client resolves at call time, and roll back by re-pointing it [34, 36, 46]. The question *may this change go live, on what evidence, and how do I undo it* is therefore answered today for one artifact type, by pointer discipline, with the evidence sitting beside the decision rather than gating it.

What remains unanswered is the same question for everything else an agent depends on — its workflows, tool definitions, MCP server configurations, and retrieval corpora — and the question of what it would cost to answer it for all of them at once. That is this paper's subject, and §11 states the delta against those platforms feature by feature rather than by category.

Three words, used deliberately differently. Because this paper distinguishes things a looser treatment would conflate, we pin the vocabulary once and hold to it. An **AI-agent resource** is something in the world an agent depends on at run time: a prompt, a workflow, a tool, an MCP server, a corpus. An **artifact** is the generic term for such a thing considered as a versioned object, used when the contrast is with *code*, which already has a release discipline. A **governed asset** is CALIBER's typed record *for* one — the abstraction this paper introduces — and only the third term is technical. Where the paper says "artifact" it is never referring to an MLflow artifact-store object; Appendix D notes that collision explicitly.

This technical report. We present CALIBER, a layered control plane for the lifecycle of AI-agent resources. CALIBER's organizing abstraction is the *governed asset*: a typed record carrying a schema-validated definition, version history, an authority model, a trace and audit trail, and — where the asset's family supports one — a live target that client code resolves at call time, so a release changes behaviour without a client deployment. Around that abstraction CALIBER arranges six layers and six reusable lifecycle modes, and composes them into a governance chain that carries a flagged production trace to a measured candidate, an enforced evaluation gate, an explicit human decision, and an audited release that records the outgoing target so rollback is a restore rather than a reconstruction.

The central claim, stated narrowly. It would be easy to claim that a layered control plane makes governance uniform across artifact types. That claim would be false, and the interesting content of this paper is why. Our actual claim is about what layering does and does not buy:

> *Adjacency in a layered architecture confers capability* availability*, not capability* inheritance. *A family placed in the asset layer obtains lifecycle behaviour and governance enforcement only by explicitly wiring them. The shared base contract is identity, history, authority, and audit; release, rollback, evaluation, and evidence are capability-specific obligations.* CALIBER *declares those obligations per family in a checked registry, but does not yet tie a declaration to the handler that discharges it (§5.1).*

We defend the per-family capability factoring while treating the remaining gap — declarations that are checked against each other but not against the handlers behind them — as technical debt (§5). A prompt declares live-target release and rollback; a test set is evidence and declares no live target; a judge is evaluatable but not releasable. We therefore give the guarantee surface in full, including the rows where it is weaker than a reader might assume (Table 1).

Contributions.

1. **A typed, per-family governance architecture.** A six-layer factoring in which the unit of governance is an asset rather than a model or trace, plus an explicit capability/guarantee matrix that separates the shared base contract from family-specific release, rollback, and evidence semantics (§4, §5).
2. **A governance chain distinguished from its implementations.** A seven-term reference shape — SIGNAL, EVIDENCE, CANDIDATE, MEASUREMENT, DECISION, RELEASE, TRACE — with an explicit account of how the concrete six-stage prompt path realizes it non-bijectively, and of what durable residue each term leaves so that a release is reconstructable from records (§4, Figure 4).
3. **A gate taxonomy that survives contact with operators.** The separation of an *enforced* candidate-advancement gate from an *advisory* per-version release verdict, and the argument that putting the enforced gate on candidate advancement rather than on release is what keeps operators from routing around the system (§7, D3 in Table 4).
4. **A stated per-loop guarantee surface for a broker-free execution model.** Claim-and-lease columns on relational rows are a known pattern [30] and collapsing the worker tier into the API process is an engineering choice rather than a research result; we claim neither as novel. What we do contribute is the discipline of stating what each of the ten loops actually guarantees — they differ, and an earlier draft of this paper wrongly described them with one number (Table 2) — together with the honest converse that in-memory rate limits become per-process and must be sized for the worker count (§5.7).
5. **Precise state ownership, including its limits.** Three stores, three owners, and the two dual-write boundaries no transaction can span. Naming those boundaries is what allows a precise audit guarantee to be stated at all (§6, Figure 7).
6. **A comparison that includes the losses.** CALIBER against tracking, composition, multi-agent, and optimizer systems along nine dimensions, with a cost column for every row including CALIBER's own (§11).
7. **A measurement protocol, offered as an artifact rather than as a result.** Appendix C specifies the questions, the baselines, the fault-injection matrix, and the analysis in enough detail to be executed by someone else. We want to be exact about its standing: an unpopulated table is

not a contribution, and we do not present it as one. The protocol is a usable artifact; the results would be the contribution, and they are absent (§10).

What this paper does not claim. CALIBER is not a composition or multi-agent orchestration framework, and does not compete with one; agents are composed elsewhere and governed here. It does not rebuild tracking: MLflow's experiments, traces, assessments, prompt registry, artifact store, and evaluation entry points are dependencies, not reimplementations [44, 46]. It ships a single environment with no `dev` → `staging` → `prod` ladder. Its local tool and MCP execution is *containment*, not isolation, and we are careful throughout about which word applies (§12). Most importantly, the quantitative evaluation in §10 is specified but not yet executed. We state that plainly and mark each unpopulated result, because a systems paper that decorates an unrun experiment with plausible numbers is worse than one that reports an empty table.

2 Motivation

2.1 A failure no single resource lifecycle closes

Consider a support agent in production. A customer asks a question about a refund window; the agent answers confidently and incorrectly, citing a policy that changed last quarter. A reviewer marks the response as bad. Every step up to this point is well tooled: the call produced a trace, the trace carries the retrieved chunks and the rendered prompt, and the reviewer's judgement is recorded as an assessment attached to that trace.

Now enumerate what a team must do to actually fix it, and what tooling exists for each step.

1. **Decide the failure is real** and not a mislabel. Tooling: a queue, if someone built one.
2. **Locate the cause** — stale retrieval, an over-confident instruction, a tool returning a wrong field, or a prompt that never told the model to check recency. Tooling: reading the trace.
3. **Assemble evidence** beyond this one case, so the fix is not overfitted to a single input. Tooling: none standard; teams hand-build spreadsheets.
4. **Produce a candidate** change. Tooling: an optimizer, if the team has adopted one, run out of band [29, 65].
5. **Measure** the candidate against the incumbent on that evidence, per dimension, not just on the reported case. Tooling: an evaluation harness [50, 54], run manually.
6. **Decide** whether to ship. Tooling: a human, informally.
7. **Release** it such that running agents pick it up. Tooling: a code deployment, because the prompt is a string in a source file.
8. **Record** what changed, on what evidence, approved by whom. Tooling: a commit message, if anyone writes one.
9. **Be able to undo it** to the exact prior state. Tooling: `git revert` plus another deployment — which reverts the prompt but not the retrieval corpus, the tool version, or the judge that scored it.

Steps 1–5 are addressed by tools that often do not share one lifecycle. For prompts, current registries cover important parts of steps 6–9: immutable versions, live labels, rollback, and permissions [34, 36, 46]. The unresolved part is applying an evidence-linked release vocabulary across the other agent resources and making the guarantee surface explicit per family. This is the gap CALIBER targets. It is not a gap in model quality or observability; it is a gap in *cross-resource release engineering for non-code artifacts*, structurally related to the gap continuous delivery [26] closed for code and that TFX [8] and MLflow [68] closed, partially, for models.

2.2 Why the obvious answers do not work

"Put the prompts in git." Version control gives history and review, and it is genuinely necessary. It does not give a live target a running agent can resolve without redeployment; it does not attach evidence to a version; it cannot express that this version passed a regression gate on that corpus; and it cannot record that an operator acting under a declared scope authorized the change. Most decisively, it binds the prompt's release to the application's release, which means the fastest possible remediation is gated on a deployment pipeline built for code.

"Use the prompt platform's evaluation features." Current prompt platforms do own versioned prompt artifacts and can link evaluations to them; pretending otherwise would erase the closest baseline. The narrower limitation is scope: those prompt lifecycles do not by themselves define compatible release evidence and rollback semantics for workflows, tools, MCP configurations, skills, and retrieval corpora (Table 7).

"Use an optimizer." DSPy, GEPA, MIPRO, OPRO, and TextGrad are real advances and CALIBER uses them [2, 29, 51, 65, 67]. But an optimizer produces a candidate; it does not produce a release. It has no notion of who may promote, what the outgoing live target was, or how to restore it. Treating an optimizer as a lifecycle solution is a category error: it is the compiler, not the deployment system.

"Automate the whole loop." This is the tempting answer and we argue it is wrong at the current state of the art, which is why CALIBER keeps a human at APPLY (D4, Table 4). A score improvement on an assembled corpus is necessary but not sufficient evidence that a change is safe: the corpus is a sample, judges are correlated with each other and with the model being evaluated [69], and the failures that matter most in agent systems — tool misuse, over-confident assertion, prompt injection reached through retrieved content [23] — are precisely the ones a metric constructed from prior failures is least likely to catch. An auto-apply threshold converts a visible, reviewable change into an invisible, correlated one.

2.3 What a solution has to provide

The scenario above yields five requirements, which the rest of the paper is organized around.

R1 — Typed, versioned artifacts. The unit of change must be a record with a schema and an immutable version history, not a string in a file (§6).

R2 — Late-bound live targets. Client code must resolve *which* version to run at call time, so remediation is decoupled from application deployment (§7).

R3 — Evidence bound to versions. Scores, judgements, and the corpus they were computed over must be attached to the version they describe, durably, so a release is reconstructable months later (§4).

R4 — An enforced gate plus a human decision. Some check must be unbypassable and some judgement must be a person's, and §7 argues that these should be different mechanisms acting at different points in the path rather than one mechanism doing both.

R5 — Audited, reversible release. The release must record what the live target *was*, so undo is a restore (§6).

R1 through R5 are not novel in the abstract — they are the standard demands of release engineering [26], artifact provenance [48, 62], and documented model governance [21, 43]. The contribution is not inventing them but discharging them over a set of artifact types that are genuinely heterogeneous, and being precise about where that heterogeneity forces the guarantees apart.

3 Background and Related Work

We group prior work by the question each body of work answers, because the groupings are not competitors — they occupy different positions in the same stack, and CALIBER depends on several of them.

3.1 ML lifecycle platforms

The ML platform literature established that the model is not the deliverable: the pipeline around it is. TFX [8] articulated production-scale lifecycle management; MLflow [68] made experiment tracking, model versioning, and artifact storage into general infrastructure; and Kubeflow [31], Airflow [5], and DVC [27] covered orchestration and data versioning. Work on automatic capture of experiment metadata and provenance [58] established that lineage has to be recorded by the platform rather than by the practitioner — a premise CALIBER inherits and extends from experiments to releases. Alongside them, a diagnostic literature described what goes wrong: hidden technical debt in ML systems [60], the engineering practices that follow [3], production-readiness rubrics [10], and documentation norms for models and datasets [21, 43].

CALIBER sits in this tradition and deliberately does not re-enter it. MLflow's experiments, traces, assessments, prompt registry, artifact store, dataset registry, and evaluation entry points are dependencies (D1, Table 4). The distinction we draw is one of *unit*: these platforms govern models and pipelines, whose deployable is a serialized artifact plus code. The artifacts that determine an LLM agent's behaviour are prompts, tool definitions, skills, retrieval corpora, and judges — and these have a property models do not, which is that the deployable is often a *selection* rather than a binary. The consequence is that the interesting release primitive is alias indirection, not artifact shipping, and none of the model-centric platforms provide it for these types.

3.2 Agent architectures and composition frameworks

A large body of work establishes how to build agents: ReAct [66] interleaving reasoning and acting, Toolformer [59] on tool invocation, chain-of-thought prompting [63], self-critique and reflection [41, 61], retrieval augmentation [37], multi-agent conversation [64], and simulated societies of agents [52]. The engineering counterpart is a set of composition frameworks — LangChain and LangGraph [32], LlamaIndex [40], CrewAI [17], the OpenAI Agents SDK [49] — plus the Model Context Protocol [4] as an emerging interoperability layer for tools and resources.

These answer *how to build*, and CALIBER is explicitly not a competitor: an agent composed with any of them is a client of a CALIBER-governed asset, resolving a live target at call time (Figure 2). The relationship is the one between a web framework and a deployment system. CALIBER does interpret workflows internally over a typed IR with 31 node types, but that IR exists to make a workflow a *governable* artifact — versioned, replayable, gated — rather than to compete on expressiveness.

3.3 Prompt and program optimization

Automated prompt search has moved quickly. APE [70] framed instruction generation as search; OPRO [65] used the model itself as the optimizer; DSPy [29] recast LLM pipelines as compilable programs with MIPRO [51] optimizing instructions and demonstrations jointly; GEPA [2] showed reflective prompt evolution competitive with reinforcement learning; and TextGrad [67] backpropagates textual feedback.

CALIBER treats these as pluggable strategies behind one seam, and exposes five implemented paths (Figure 10). The architectural observation we would emphasize is that this literature has an implicit precondition it does not supply: an optimizer needs a metric, a corpus, and a baseline.

Where those come from, whether they are versioned, and whether the resulting candidate may be released are outside its scope — and are exactly what §7 is about. This is why we characterize optimizers as compilers rather than as lifecycle solutions.

3.4 Evaluation, judges, and human alignment

Holistic benchmark suites [38] established multi-dimensional evaluation as the norm; LLM-as-judge methods made per-example scoring affordable [39, 69]; RAGAS [18] specialized this for retrieval-augmented systems; and harnesses such as promptfoo [54] and OpenAI Evals [50] made offline suites routine; and Guardrails [24] moved a subset of these checks inline, as runtime validators rather than offline scorers — a complementary position, since a runtime validator constrains one call while an offline suite characterizes a version. The known hazards are equally well established — judges are biased toward verbosity and toward outputs from related models, and agreement with human labels must be measured rather than assumed, which is why CALIBER records inter-rater agreement statistics for authored judges [15, 19].

The gap CALIBER addresses is not scoring quality but *binding*. In the harness model, a score is a number in a report. In CALIBER, a score is a durable record attached to a candidate version, admissible as evidence at a gate, and carried into a human review (Figure 4). The same computation becomes a different kind of object when it is bound to the artifact it describes.

3.5 Observability and prompt-lifecycle platforms

LangSmith [33], Langfuse [35], and Arize Phoenix [7] collect traces, run offline and online evaluations, and present the results, converging on OpenTelemetry and its GenAI semantic conventions [13, 14]. They are the closest neighbours to CALIBER, so it matters to state the boundary accurately rather than sharply.

Two of them, together with MLflow's Prompt Registry, have grown past observability into prompt lifecycle management, and a reader should not carry the older mental model into this paper. As of 2026-08-03 all three provide versioned prompts, a named pointer that client code resolves at call time — MLflow aliases [46], LangSmith commit tags including a reserved `production` [34], Langfuse labels [36] — rollback by re-pointing, and role restrictions on who may move the pointer. MLflow also ships prompt optimization driven by a dataset and scorers [45]. CALIBER's mechanisms for prompts are substantially the same mechanisms, and this paper does not claim otherwise.

What we did not find documented in any of them is a second governed artifact type beyond prompts and models, or an evaluation result acting as a *precondition* on the pointer move rather than as an annotation beside it. Those two gaps, not an empty ecosystem, are what §11 argues CALIBER occupies, and Table 7 states the comparison row by row against dated sources.

3.6 Isolation, policy, and provenance

CALIBER executes operator-authored tool code and connects to third-party MCP servers, which places it in a well-mapped security landscape. The foundational principles are Saltzer and Schroeder's least privilege and fail-safe defaults [56]; the authority model is role-based access control [57] with the integrity and separation-of-duty concerns Clark and Wilson formalized [11]. Strong isolation for untrusted code is a solved problem given the right substrate — microVMs such as Firecracker [1], or unprivileged sandboxing via bubblewrap [16] — and policy engines such as Open Policy Agent [12] externalize authorization decisions. On the provenance side, in-toto [62] and SLSA [48] define what it means for an artifact's history to be attestable.

Two points of intellectual honesty follow, and we return to both. First, CALIBER's local tool and MCP execution is *containment* — allowlists, a sanitized environment, a private working di-

rectory, process timeouts — and not isolation in the Firecracker sense; we use the two words distinctly throughout (Figure 5). Second, CALIBER's provenance anchors are internal records, not cryptographic attestations in the in-toto sense, and closing that gap is future work (§13). Prompt injection through retrieved or tool-returned content [23, 53] is a threat CALIBER's governance model constrains — by bounding what a capability may do — but does not solve.

3.7 Distributed-systems foundations

CALIBER's execution model rests on old results. Its durable queues are status, claim, and lease columns on relational rows, which is the transactional-outbox and lease pattern rather than anything new [22, 30]. Its two cross-store crossings cannot be made atomic, so they are compensated rather than transactional, in the spirit of sagas [20] and of Helland's argument that distributed transactions are the wrong default for scalable systems [25]. Its SLO and incident surface follows standard site-reliability practice [9]. We claim no novelty in any of this; we claim only that applying it to *agent-resource release* is what turns an observability tool into a control plane, and §8 records where each borrowed pattern charges its price.

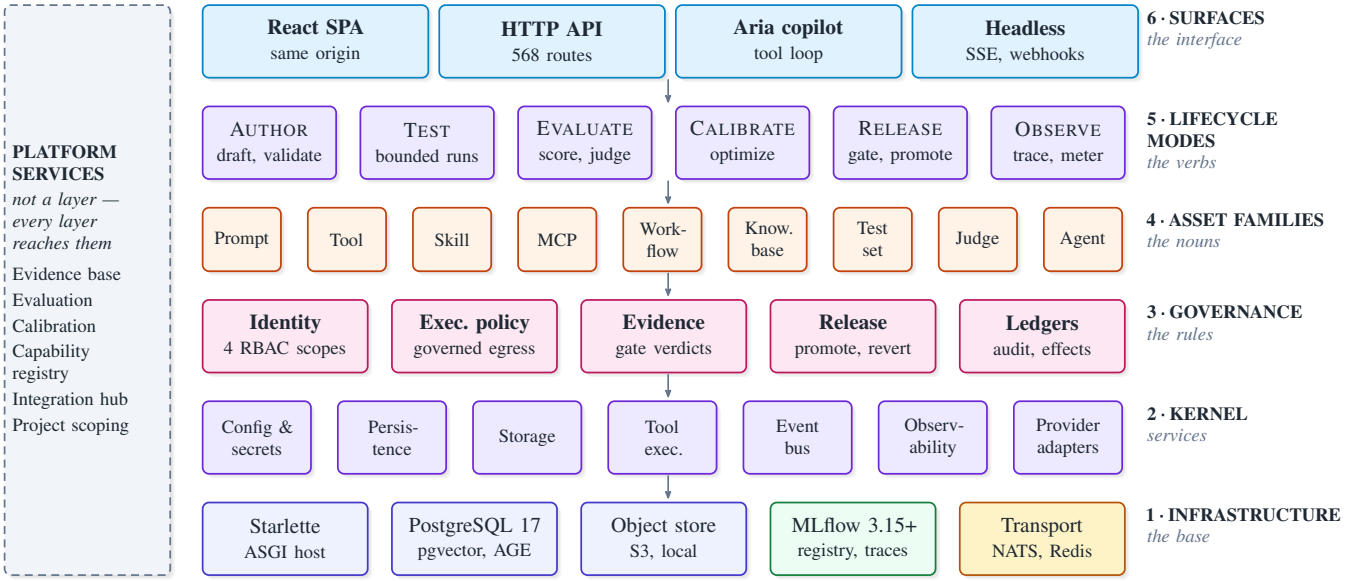


Figure 1: The CALIBER stack. Six layers, read bottom-up: L_1 (infrastructure) carries L_2 (kernel); kernel services obey L_3 (governance); the substrate governs L_4 (asset families) — the nouns; L_5 (lifecycle modes) — the verbs — act on those nouns; and L_6 (surfaces) exposes the result. The dashed left rail holds platform services that cut across all six layers rather than occupying one. Adjacency in this diagram is *capability availability, not inheritance*: a family in L_4 obtains L_5 behaviour and L_3 enforcement only by explicitly wiring them, which is why the guarantees in Table 1 differ per row instead of being uniform. That distinction is the paper's central architectural claim (§5).

4 System Overview

CALIBER is one ASGI application, plus a React single-page application it serves from the same origin, plus the state it owns. Figure 1 gives the layered view and Figure 2 the runtime view; this section introduces the three concepts a reader needs before either diagram is useful.

4.1 Design principles

Ten principles constrain what CALIBER builds. They are ordered, and the order is the load-bearing part: purpose and trust first, then capability and architecture, then operational quality, and finally technology choices and longevity. Where two principles conflict, the lower-numbered one wins — a governance guarantee outranks a convenience, and an audit trail outranks a shortcut. Stating the tie break is what makes the list a design constraint rather than a description.

1. **Governed agentic workflows.** Governance, security, compliance, and policy enforcement at the core.
2. **Progressive autonomy.** Support the path from human-led processes to fully autonomous agent-driven execution.
3. **Auditability and observability.** Every workflow, decision, and agent action traceable, transparent, and explainable.
4. **Evaluation by design.** Testing, verification, validation, benchmarking, calibration, and optimization built in from the start.
5. **Open and extensible.** Integrate with enterprise systems, models, tools, and external agent platforms through open APIs and SDKs.

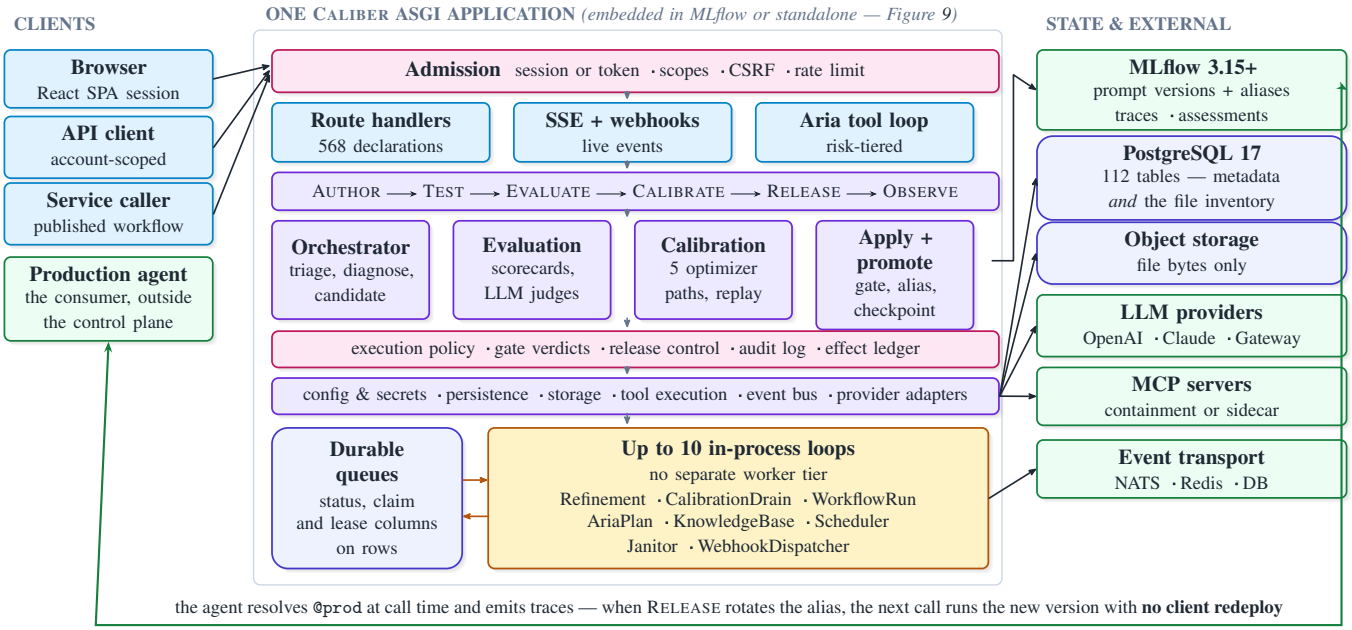


Figure 2: High-level architecture and component interaction. Every account client and service caller enters through one admission stage; L_5 (lifecycle modes) drives four control-plane service groups; and the kernel is the only tier that touches state or external systems. Two properties are load-bearing for the rest of the paper. First, CALIBER is a *sibling* surface to MLflow, not a proxy in front of it: each owns its own store, which is what makes the reconciliation boundary in Figure 7 necessary. Second, there is no separate worker tier — queued work is arbitrated through claim and lease columns on relational rows, so one process both serves requests and drains queues. The paired arrows between the queue and the loop group are a consumer's atomic claim and the status transition it writes back (Algorithm 1). The outer lane is the only path that leaves the platform entirely, and it is why a release needs no client deployment.

6. **Modular and composable.** Loosely coupled components that are reusable, flexible, and easy to extend.
7. **Scalable and reliable.** Enterprise-grade performance, resilience, and operational reliability.
8. **Open-source first.** Prefer proven open-source building blocks with strong community support.
9. **Developer and user friendly.** A clear, productive experience for both developers and business users.
10. **Future ready.** Design for evolving AI capabilities, orchestration patterns, and enterprise needs.

Two of these are worth reading against the rest of this paper rather than taken on trust. Principle 8 constrains *how* the others are met and does not inherit their guarantees: §8 records reusing MLflow rather than owning the substrate, and §6 states the price — the audit guarantee holds for database-resident mutations and not across the MLflow boundary. Principle 4 is the one the evaluation in §10 is designed to test rather than assert, and §12 records where per-family coverage stops being uniform.

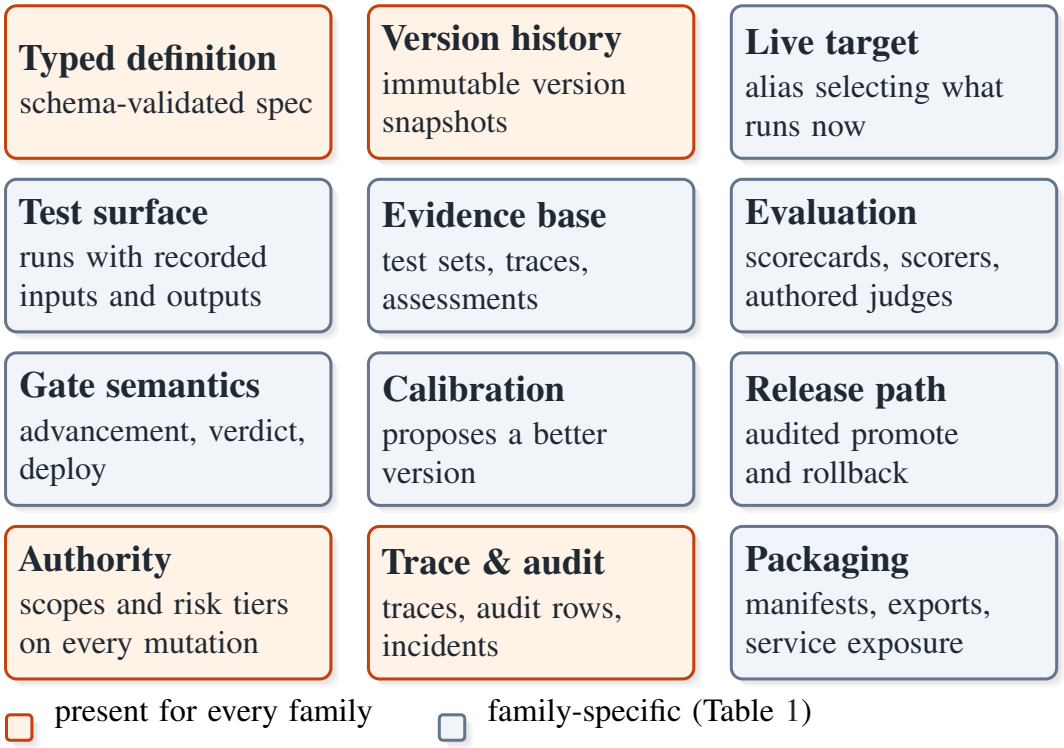


Figure 3: Anatomy of a governed asset: the unit of value the whole platform exists to produce. Twelve facets are *possible*; a family implements and explicitly wires only those its idiom supports. The four solid facets hold for every family because they are what makes a record governed at all — a typed definition, version history, an authority model, and a trace and audit trail. The eight muted facets do not transfer: test sets and judges are evidence and scoring assets with no live target and no release path, tools expose family history without a live alias, and gate semantics differ per family rather than being one cross-artifact policy. What a governed asset is *not*, stated plainly: not a prompt file in version control, not a dashboard over traces, and not an agent framework. It is a typed record carrying enough version, evidence, and release metadata that a change to it is reviewable after the fact.

4.2 The governed asset

The unit of value is the governed asset: a typed record that carries enough metadata that a change to it is reviewable after the fact. Figure 3 enumerates twelve possible facets. Four hold for every family, because they are what makes a record governed at all:

- a **typed definition** — a schema-validated specification that is the source of truth for the asset, not a rendering of it;
- **version history** — immutable snapshots, or immutable registry versions held externally;
- an **authority model** — which of the four RBAC scopes may read, mutate, and release it;
- a **trace and audit trail** — the durable record of what was done to it and by whom.

The remaining eight — live target, test surface, evidence base, evaluation, gate semantics, calibration, release path, packaging — are *family-specific*, and that asymmetry is the subject of §5.

It is worth being explicit about what a governed asset is not. It is not a prompt file under version control: version control gives history but no live target, no bound evidence, and no authority model. It is not a dashboard over traces: a dashboard describes behaviour but owns no artifact.

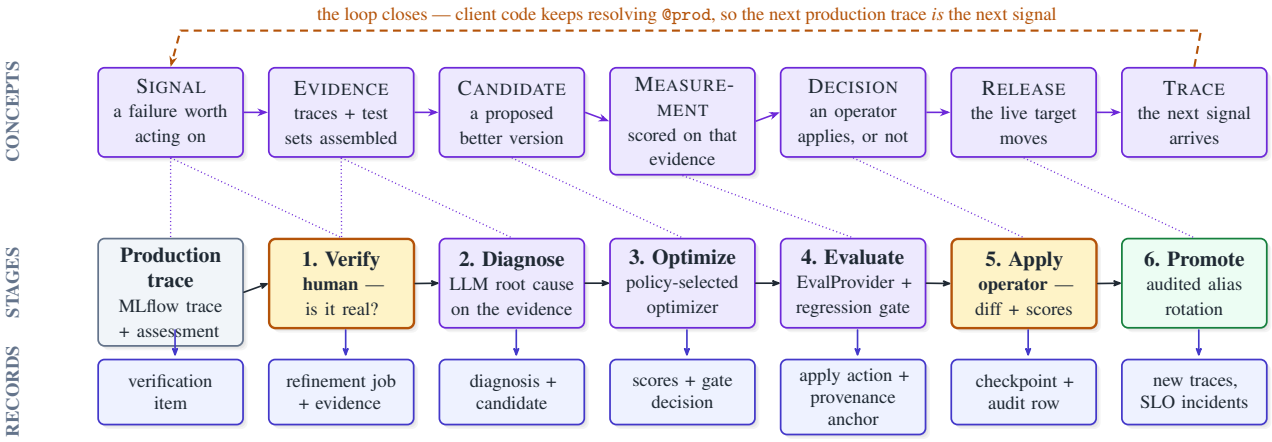


Figure 4: The governance chain in three registers. The top row is a *reference shape*, not a worker pipeline: it names seven concepts a refinement-capable asset family may occupy. The middle row is the concrete prompt path — the fullest realization of that shape — with six numbered stages and exactly two human decisions. The dotted correspondences are deliberately not one-to-one, and the mismatch is the figure's argument: EVIDENCE has no stage of its own, because evidence assembly is folded into the transition from VERIFY toward DIAGNOSE; SIGNAL spans both an incoming production trace and the human confirmation that the failure is real; and TRACE is simultaneously the seventh term and the input to the next iteration. Reading the seven terms as seven worker stages is the single most common misreading of this architecture. Other families reuse the shape without inheriting its guarantees: a workflow is measured by manifest replay, a tool by a revision-fenced deterministic suite, a knowledge base by retrieval calibration (Table 1). The bottom row is what separates this from an observability dashboard — every term deposits durable state, so a release is reconstructable from records rather than from recollection.

surfaces · control plane · governance · asset families · external systems · asynchronous · durable state

And it is not an agent: agents are composed in a framework of the team's choosing and are *clients* of governed assets.

4.3 Six lifecycle modes

Six verbs act on those nouns, and they are deliberately few: AUTHOR (draft, render, compile, validate), TEST (bounded runs against fixtures), EVALUATE (scorecards, deterministic scorers, authored LLM judges, review queues), CALIBRATE (propose a measurably better version), RELEASE (gate, promote, roll back), and OBSERVE (trace, meter, incident, replay).

The modes are reusable concepts, not a state machine every asset traverses. A test set is authored, tested against, and observed, but is never released — it is the evidence others are released against. A judge is authored and evaluated for human agreement, but has no live target. Enumerating the modes separately from the families is what makes the coverage matrix in Table 1 expressible at all: without that separation there is only a single undifferentiated notion of "supported."

4.4 The governance chain

The chain is what makes a change reviewable, and Figure 4 presents it in three registers. The upper register names seven concepts: SIGNAL (a failure worth acting on), EVIDENCE (traces and examples assembled into a corpus), CANDIDATE (a proposed better version), MEASUREMENT

(scored against that corpus), DECISION (an operator applies, or does not), RELEASE (the live target moves), and TRACE (the next signal arrives).

We stress a point the figure is drawn to make and that is routinely misread: these are *seven concepts, not seven stages*. The concrete prompt path has six numbered stages, and the correspondence to the seven terms is not bijective. EVIDENCE has no stage of its own — evidence assembly is folded into the transition from VERIFY toward DIAGNOSE. SIGNAL spans both an incoming production trace and the human confirmation that the failure is real. TRACE is simultaneously the seventh term and the input to the next iteration. A reader who counts stages and finds six where the text says seven has found this non-bijection, not an inconsistency.

The lower register of Figure 4 is the part that distinguishes this architecture from an observability pipeline. Each term deposits durable state: a verification item, a refinement job with its assembled evidence, a diagnosis and candidate artifact, scores with an enforced gate decision, an explicit apply action with a provenance anchor, a rollback checkpoint with an audit row, and new traces. Six weeks later, the question "why is the prompt like this, and what did we know when we changed it" is answerable from records rather than from recollection. That property — not any individual mechanism — is what CALIBER exists to provide.

4.5 Reading the two architectural diagrams

Figure 1 reads bottom-up. L_1 (infrastructure) is the substrate CALIBER runs on, all of it swappable by configuration. L_2 (kernel) holds the modular services built at application startup. L_3 (governance) is the substrate of shared primitives that asset paths wire explicitly. L_4 (asset families) holds the nine governed families — the nouns. L_5 (lifecycle modes) holds the verbs. L_6 (surfaces) exposes the result. The dashed left rail holds platform services — the evidence base, evaluation, calibration, the capability registry, the integration hub, project scoping — that cut across all six layers rather than occupying one.

Figure 2 reads left-to-right and shows the runtime instead. Every account client and service caller passes through one admission stage; lifecycle modes drive four control-plane service groups; the kernel is the only tier that touches state or external systems; and queued work is drained by up to ten in-process loops, six of them arbitrated through claim and lease columns on relational rows and the rest correct by idempotence (Table 2). Two properties are load-bearing for everything that follows. First, CALIBER is a *sibling* surface to MLflow rather than a proxy in front of it — each owns its own store, which is why the reconciliation boundary described in §5.7 exists and cannot be designed away. Second, there is no separate worker tier, which is a deliberate trade with consequences we spell out in §5.7.

5 Multi-layer Architecture

This section defends the layering, states its central limitation as a positive design claim, and then walks the layers.

5.1 Availability, not inheritance

Layered architectures invite an inference: if governance is a layer beneath the asset layer, then every asset is governed by it. In CALIBER that inference is false, and we consider saying so precisely to be more useful than papering over it.

Placing a family in L_4 (asset families) makes the L_3 (governance) primitives and the L_5 (lifecycle modes) behaviours *available* to it. It does not apply them. Each family must explicitly wire its adapters, routes, authorization checks, evidence handling, and tests. Adjacency confers availability; only wiring confers enforcement.

This is not an implementation shortfall awaiting a refactor, and the argument is worth making carefully because a reviewer will reasonably suspect otherwise. Consider what a uniform cross-artifact release contract would have to mean across three families CALIBER governs:

- For a **prompt**, a live target is an MLflow registry alias, and release means rotating it. Rollback means restoring the alias to the target it previously held.
- For a **test set**, there is no live target at all. A test set *is* evidence; "releasing" one is not a meaningful operation, and forcing it to have a release path would produce a field nothing reads.
- For a **judge**, the artifact is a scorer. It is referenced by token wherever a scorer is accepted. It has neither a live target nor an evidence base of its own; what it has is a measured agreement rate with human labels [15].

Any *single* contract all three satisfy is vacuous — it can require nothing stronger than "has an identifier." Conversely, a single contract strong enough to be worth enforcing on prompts would exclude test sets and judges from the platform, which would make it less useful rather than more principled.

A third design, which we did not take. Presenting that as an exhaustive choice would be a false dichotomy, and an earlier draft of this paper did exactly that. There is a well-known third option: a *thin base contract* that every governed asset satisfies — identity, immutable version history, an authority check, an audit record — plus a set of orthogonal *capability interfaces* that only the families possessing them implement, such as `Releasable`, `Rollbackable`, and `EvidenceBearing`. This is the ordinary answer to heterogeneous types in a typed system, and it is strictly better than what CALIBER does today in one specific and important respect. A family that declares `Releasable` could be *required* to supply a live-target resolver, a checkpoint writer, and a rollback path, and the requirement could be checked by the type system or by a registry projection at start-up rather than by a reviewer reading a table. The per-family variation would remain — it is intrinsic, since a test set genuinely has no live target — but the *obligation* would be enforced instead of documented, and the failure mode described below would become impossible to introduce rather than merely discouraged.

We did not take that design in full, and the reason is historical rather than principled: CALIBER's families were wired one at a time as they were needed, and by the time the common shape was visible the wiring already existed in 9 places. Retrofitting the interfaces is the natural next step and §13 records it as such.

What does exist is the design's data half. Each family's capabilities are declared in one machine-readable registry — kind, history idiom, live-target shape, promotability, rollback *mech-*

anism, evidence, gate mode, calibration — over a closed vocabulary, and the declarations are checked against each other when the module loads. A family cannot declare itself promotable with no live target to promote, cannot carry a rollback flag that disagrees with its rollback mechanism, and cannot be an evidence or scoring asset that is nonetheless deployable; each of those is a contradiction the process refuses to start with rather than a row that renders like any other. Table 1 is generated from that registry, so the table and the implementation cannot state different guarantees.

We are precise about what this does not establish, because the gap is the interesting part. Checking a declaration against its siblings is not checking it against its wiring: nothing yet requires a family declaring a rollback mechanism to supply a handler that implements it. The registry makes the obligation *stateable and self-consistent*; the capability interfaces above are what would make it *discharged*. Carrying the mechanism rather than a boolean is what makes the difference visible at all — four families declare rollback and no two mean the same thing by it, so a client given only the flag would infer one guarantee from four.

What we therefore claim. The claim this paper can defend is narrower than "uniformity is impossible." It is that *uniform guarantees* are impossible across genuinely heterogeneous families, because the guarantees differ in kind; and that *uniform obligations* are possible but require a mechanism CALIBER has only in part. CALIBER's current position — per-family guarantees *declared* in a checked registry and rendered from it, but not tied to the handlers that implement them — sits between the two defensible designs, and we would characterize it as a state of the implementation rather than a finding about architecture.

What the substrate does buy. Rejecting inheritance is not the same as rejecting shared structure, and it would be a misreading to conclude the layering does no work. The substrate provides one implementation of session verification and scope resolution that every account route uses; one audit-record helper whose transactional semantics are stated once (§6); one egress policy for governed HTTP and MCP transports; one storage abstraction; one event bus. A new family inherits none of these *automatically*, but it also does not reimplement any of them — wiring is a call, not a rewrite. The cost of adding a family is proportional to its distinct semantics rather than to the whole governance surface, which is precisely what layering is supposed to buy. What layering cannot buy is the claim that the wiring was done, and the honest response to that is a table, not a paragraph.

The residual risk, stated. The failure mode this design admits is a family that appears in L_4 (asset families) without having wired a primitive a reader assumes it wired. The shared version-history component is the clearest instance: it is mounted for prompts, workflows, knowledge bases, skills, and tools through per-artifact adapters, and *sharing the component does not share the semantics*. A reader who sees the same panel in five places may reasonably infer five equivalent rollback guarantees, and will instead find five distinct semantics: an alias restore, a checkpoint-stack pop, a derivation from activation history, a restore-as-a-new-version, and — for tools — no rollback at all (Table 1). We consider this the most likely source of operator misunderstanding in the system. The mitigation is that the mechanism, not merely a boolean, is what the capability registry carries and what the capabilities endpoint publishes — so a client is able to render the five as five. It remains a mitigation rather than a fix: nothing compels a client to read the field, and nothing yet checks that the declared mechanism is the one the handler performs (§13).

5.2 L_1: infrastructure

The base is the swappable substrate: a Starlette ASGI process host; PostgreSQL 17 with pgvector for approximate nearest-neighbour search [28, 42] and Apache AGE for property-graph

Table 1: What each asset family actually guarantees. The nine families are not even the same *kind* of thing: six are authored runtime assets, test sets and judges are evidence and scoring assets rather than things one deploys, and *agent* is the anchor record that verification items, refinement jobs, and approvals hang off. Sharing a substrate does not make the guarantees uniform, and this table is the refutation of that inference. Note in particular that the shared version-history component is mounted for five families through per-artifact adapters — *sharing the component does not share the semantics*.

Family	History & liveness	Gate semantics	Release / rollback	Calibration idiom
Prompt	immutable MLflow registry versions behind an alias such as `@prod`	enforced advancement to `candidate_ready` advisory per-version verdict	operator- or admin-scoped; records and restores the outgoing alias target	provider optimizer + EvalProvider
Workflow	editable drafts promoted to published version rows; deployment aliases select one	enforced readiness enforced deploy gate, optimistic alias check	rollback pops the deployment's checkpoint stack	manifest replay
Knowledge base	immutable build versions behind `active_version_id`	none	audited activation; rollback derives the prior build from history	retrieval-quality calibration
Skill	mutable current record plus immutable snapshots	enforced advancement none release gate	rollback restores the prior snapshot as a *new* version	agent-free optimizer path
Tool	separate `(name, version)` rows with lifecycle status	none	read-only history; *no* live alias	revision-fenced deterministic suites
Test set	version counter plus per-example validity intervals	n/a it *is* the evidence	none no live alias or rollback	n/a
MCP server	mutable managed definitions with discovered inventories and audited edit history	production workflow preflight	none no version rollback; connection and policy controls are fail-closed	connection and policy tests
Judge	operator-authored, reusable by `Judge.<id>` token	n/a it *is* a scorer	n/a	Human-alignment agreement (κ)
Agent	the anchor record that items, jobs and approvals attach to	n/a	`enabled` is the pause/resume lever the workers read	n/a

A none or n/a chip marks a facet the family does not implement — an architectural fact about that row, not a measurement this paper omitted. enforced is unbypassable; advisory is filed as evidence and blocks nothing.

queries [6], with SQLite supported for lightweight development; object storage over local or S3-compatible backends; MLflow 3.15 or later; and an event transport that may be in-process, NATS, Redis, or the database. Every element is selected by configuration, and the layer's sole architectural obligation is that nothing above it names a concrete backend.

5.3 L_2: kernel services

Core application-lifetime dependencies are constructed in the application factory and held on shared application state: configuration and the secret store, persistence, the storage service, the tool-execution boundary, the event bus and webhook dispatch, the observability core, and the provider adapters. Some feature and runtime services are constructed lazily or per operation.

One boundary here matters more than the rest, and it is a rule about code ownership rather than about mechanism: *feature modules do not own application bootstrapping*. Core lifecycle ownership stays in the application factory even where request-scoped services are built later. Without this rule a control plane accretes import-time side effects until startup order becomes load-bearing and untestable.

5.4 L_3: the governance substrate

Five groups of shared primitives: identity and authority (sessions, the four scopes, CSRF, rate limiting); execution policy (governed HTTP and MCP egress, MCP containment, environment classification); evidence and verdicts (gate verdicts, regression detection, impact analysis); release control (apply, promote, rollback checkpoints); and ledgers (the audit log, the effect ledger, redaction).
Figure 5 details the admission and authority path. Per §5.1, these are available to every family and applied by those that wire them.

5.5 L_4: the asset families

Nine families, and they are not the same *kind* of thing — a distinction Table 1 makes concrete. Six are authored runtime assets (prompt, tool, skill, MCP server, workflow, knowledge base). Two are evidence and scoring assets rather than things one deploys (test set, judge). One is an anchor record that verification items, refinement jobs, and approvals attach to (agent). Reading all nine as deployable artifacts is the second most common misreading of this architecture, after reading the chain's seven terms as seven stages.

5.6 L_5 lifecycle modes and L_6 surfaces

The modes are described in §4. The surfaces are the React SPA at a same-origin path; a served management API spanning 568 route declarations across 63 modules, with a machine-generated OpenAPI document; a lightweight remote Python client SDK (`caliber-sdk`) plus a thin operator CLI (`caliberctl`) over that contract; the Aria copilot's permissioned agentic tool loop; and headless access by service token with server-sent events and webhooks. The architectural property of this layer is that both deployment topologies expose an identical surface (Figure 9), so the topology choice is an operations decision rather than a feature one.

5.7 Control flow, data flow, and the execution lifecycle

Control flow. A request is admitted, authorized, and dispatched to a route handler. Bounded validation and most durable mutations run inline. Work that is long-running or explicitly queued is written to a durable queue — status, claim, and lease columns on relational rows — and drained

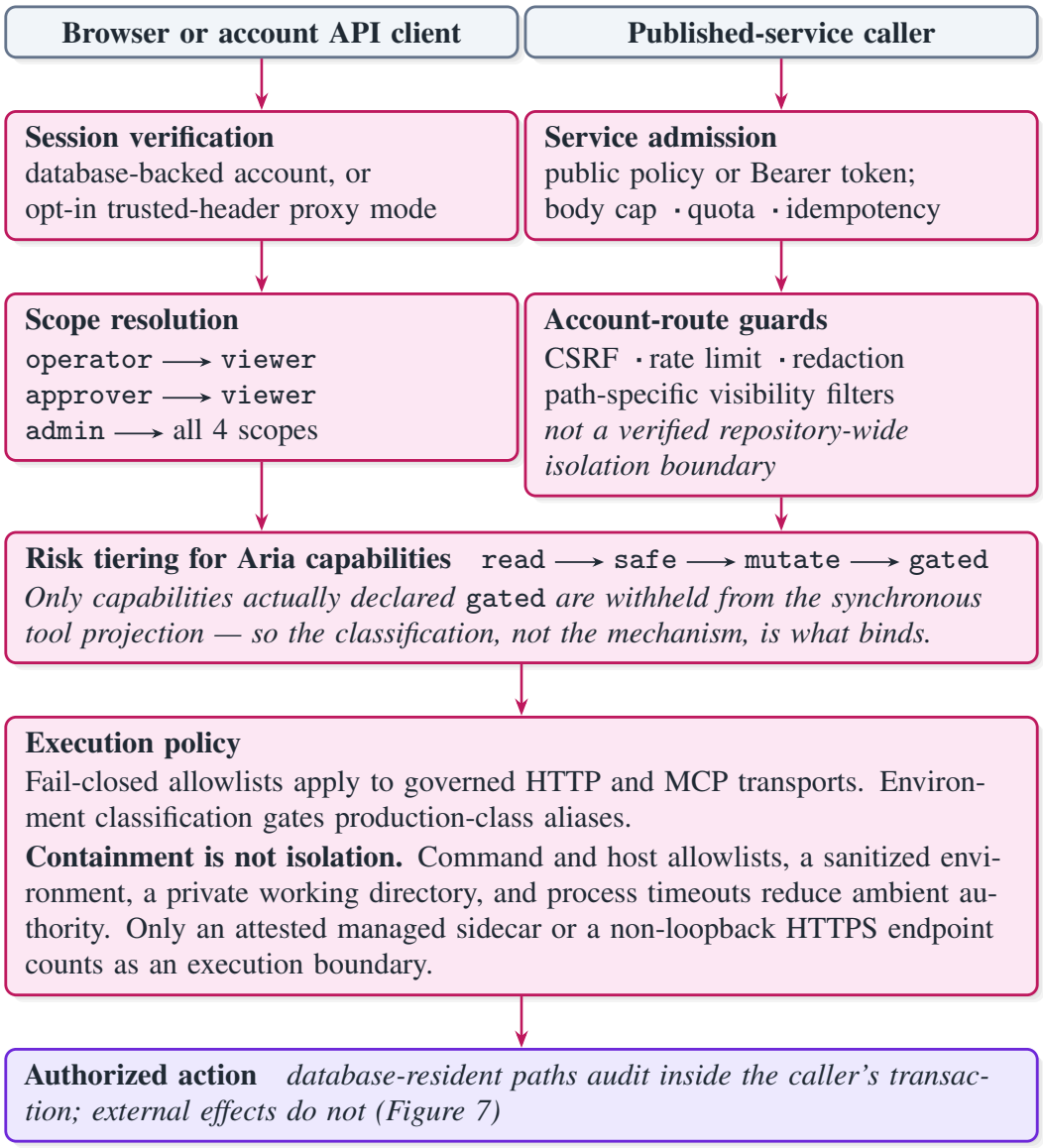


Figure 5: The trust boundary. Two entry classes with genuinely different admission rules converge on one scope lattice: `operator` and `approver` are sibling scopes that each imply `viewer`, and `admin` implies all 4. Two distinctions here are load-bearing and are routinely elided elsewhere. First, a capability is withheld from Aria's synchronous tool projection only if it is actually declared `gated`; the mechanism is sound, but the protection is only as good as the classification. Approval and publication are gated today; contract tests guard that boundary while the registry remains incomplete. Second, the local stdio MCP controls are *containment*, not a sandbox: they reduce ambient authority without establishing an execution boundary, and only an attested sidecar or a non-loopback HTTPS endpoint qualifies as one. The shipped tool runner likewise retains ambient host filesystem and network authority, so untrusted authors require an operator-supplied OS-enforced backend (§12).



Figure 6: The execution model. Bounded validation and most durable mutations run inline in request handlers; explicitly queued or long-running work goes to up to 10 in-process loops, with no separate worker tier. The split between the two loop groups is the operationally important part. Claim-based consumers compete for rows atomically (Algorithm 1), so no two workers ever hold the same row — but their *recovery* policies differ, and Table 2 gives the resulting delivery semantics loop by loop. The scheduler is idempotent by a minute-bucketed key backed by a unique partial index, which makes duplicate fires impossible rather than merely unlikely; the janitor and the release reconciler are idempotent sweeps; Aria's plan worker polls without a claim lease. The bottom box is the one place where the model is genuinely per-process, and it is stated here rather than left to be discovered in production.

Algorithm 1 The claim, as the loops write it. The arbitration is the conditional `UPDATE` on lines 4–7: two workers may both select row r, but only one `UPDATE` finds it still `queued`, and the loser sees `rowcount` $= 0$. What this establishes is *concurrent mutual exclusion* — no two workers own a row at the same instant. It does not establish exactly-once execution across a crash, because what happens to a row whose owner dies is a per-loop recovery policy, not a property of the claim (Table 2).

procedure *claim*

in: Q — a queue table with a *status* column and, on some loops, *claimed_by* and *last_heartbeat_at*

in: w — this worker's identity

invariant: no two workers hold the same row in `running` at the same instant, for any number of replicas

```
1  r ← SELECT one from Q where status = queued order by enqueued_at
2  if r = ⊥ then wait; return ⊥
3
4  n ← UPDATE Q SET status = running, claimed_by = w
5      WHERE id = r.id and status = queued                    // the arbitration
6  commit
7  if n = 0 then return ⊥                                     // another worker won it
8
9  for each step in stages(r) do
10     run(step); persist(step); commit
11     if Q has last_heartbeat_at then touch it
12 status ← done; commit
```

invariant: After the owner of a running row dies, the row's fate is decided by that loop's recovery policy. The refinement janitor marks it `failed`; the workflow-run worker requeues it on lease expiry; the calibration drain records no lease and so has no automatic recovery at all. Table 2 is the authority, and the differences are why this paper no longer states a single delivery guarantee for "the queue".

by an in-process loop. There is no broker in the arbitration path and no separate worker tier (D5, Table 4).

Algorithm 1 gives the arbitration. The mutual-exclusion argument rests on one conditional `UPDATE`: two consumers may select the same row, but only one finds it still `queued` when it writes, and the loser sees a zero row count. Nothing else in the drain loop needs to be atomic, which is why the property holds for an arbitrary number of replicas without coordination between them.

That property is *concurrent mutual exclusion*, and it is narrower than the exactly-once execution an earlier draft of this paper claimed. The two come apart after a crash, because what happens to a row whose owner died is a per-loop recovery policy rather than a consequence of the claim. Table 2 states it per loop, and the loops genuinely differ: the refinement janitor marks a stale job `failed` rather than requeueing it, so that work does not resume anywhere; the workflow-run worker requeues on lease expiry and resumes from a checkpoint; the calibration drain records no lease at all, so a row claimed by a dead worker stays `running` until an operator abandons it or creates an explicit lineage-linked retry. Aggregating these into one delivery guarantee would be wrong in both directions at once.

The ten loops divide into two groups by arbitration discipline, and Figure 6 draws them differently because conflating them is an operational hazard. Six are *durably arbitrated*: claim-based consumers compete for rows atomically, so replicating the process is safe at any worker count. Four are correct by a different argument: the workflow scheduler is idempotent by a minute-bucketed key backed by a unique partial index, which makes duplicate fires impossible rather than merely unlikely; the janitor is an idempotent sweep over jobs past their heartbeat timeout;

Table 2: What each background loop guarantees. The claim predicate (Algorithm 1) is shared, and it establishes concurrent mutual exclusion for all of them. Recovery after the owner dies is *not* shared, and that is what determines delivery semantics — which is why this paper states them per loop rather than claiming one number for "the queue". A dash marks a mechanism the loop does not have, not one we did not measure.

Loop	Lease	Recovery after owner death	Delivery semantics
Refinement	heartbeat	janitor marks the job `failed` after a stale heartbeat; it is *not* requeued	at-most-once; a crash loses the job's progress and needs operator action
WorkflowRun	lease + heartbeat	lease expiry requeues the run, which resumes from its last checkpoint	at-least-once at the step boundary; the effect ledger suppresses repeats
Calibration-Drain	none	none automatic: a row stays `running` until an operator abandons it or creates a new lineage-linked retry	at-most-once per job; retries are explicit new jobs
Knowledge-Base	claim	re-ingest is idempotent per source, so a repeat is absorbed	at-least-once, idempotent
Webhook-Dispatcher	claim	redelivery with settlement and a dead-letter path	at-least-once, by design
Workflow-Scheduler	none	idempotent by a minute-bucketed key behind a unique partial index	exactly-once *firing*; the work it enqueues then follows WorkflowRun
Janitor	none	an idempotent sweep; repeating it changes nothing	idempotent
AriaPlan	none	polls for plans parked on settled jobs; no claim is taken	at-least-once resume
Release-Reconciler	none	none needed: the next tick re-observes the alias, and a row it cannot settle from the recorded before/after versions is left for a human	idempotent convergence; `prepared` rows are never touched
Workspace-Import	claim + heartbeat	lease expiry marks the job for reconciliation, not requeued automatically	at-most-once per attempt; a retry safely replays the already-committed revision

External effects are a separate question from job delivery. The workflow effect ledger records an `in_progress` *external call whose outcome after a crash is* indeterminate *— neither confirmed nor safely repeatable — and surfaces it as such rather than guessing (§12).*

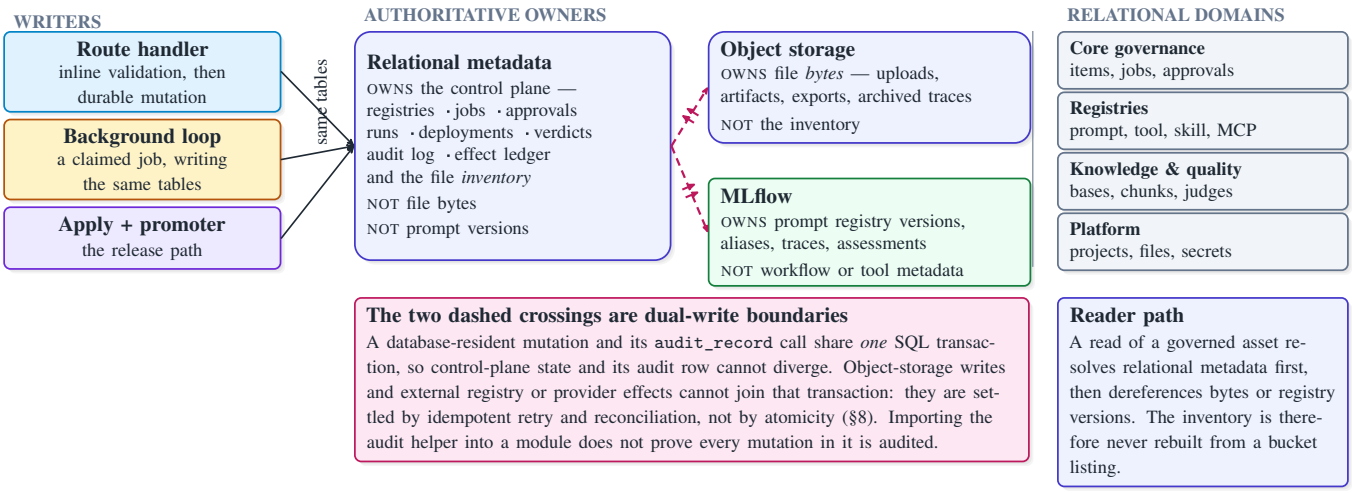


Figure 7: Data flow and state ownership. Route handlers, background loops, and the release path all write the same relational tables — which are authoritative for the control plane *and* for the file inventory. Object storage owns file bytes and nothing else; MLflow owns prompt registry versions, aliases, traces, and assessments. Solid edges are transactional. The two dashed edges are the architecture's honest limit: no transaction spans CALIBER's database and an external store, so those crossings are reconciled rather than made atomic. Naming them explicitly is what lets the audit guarantee in §6 be stated precisely instead of gestured at — database-resident mutations and their audit rows share one transaction, and nothing wider is claimed. The right-hand column is a partition of the relational store, not a fourth store.

the release reconciler is an idempotent sweep that settles a release operation only when the observed alias matches a version the intent already recorded; and Aria's plan worker polls without a claim lease.

The honest converse is that two mechanisms are genuinely per-process. The failed-login throttle is an in-memory sliding-window counter and the API rate limiter an in-memory token bucket, each behind a thread lock. Because every process runs its own full set of loops, a four-worker deployment has four independent budgets and the effective ceiling scales with the worker count. Operators must size these limits accordingly or pin a single worker. We state this in the architecture section rather than the limitations section because it is a property of the design, not a defect in it.

Data flow and state ownership.

Three stores, three owners (Figure 7). Relational metadata is authoritative for the control plane *and* for the file inventory. Object storage owns file bytes and nothing else — never the inventory. MLflow owns prompt registry versions, aliases, traces, and assessments, and does not own workflow, tool, or skill metadata.

Two crossings cannot be made atomic, and naming them is what allows a precise guarantee to be stated. A database-resident mutation and its audit-record call share one SQL transaction, so control-plane state and its audit row cannot diverge. Object-storage writes and external registry or provider effects cannot join that transaction; they are settled by idempotent retry and reconciliation in the spirit of compensating transactions [20, 25]. Two corollaries follow that a careful reader should hold onto: importing the audit helper into a module does not prove every mutation in that module is audited, and a failed dual write can leave orphaned bytes that a sweeper must reclaim (D7, Table 4).

Algorithm 2 The client-side read path — the only CALIBER code on the serving path, and therefore the entire surface E1 measures. Late binding lives on line 2: the client names an alias, never a version, which is what decouples remediation from deployment. A TTL cache can satisfy line 2 without an HTTP call. Line 5 is the property that makes the resolver's failure mode acceptable: a resolver that cannot reach the control plane serves the last known version rather than failing the call.

procedure *resolve*

in: a — an asset identity; α, an alias such as `@prod`

out: *the definition to execute now*

1 $k \leftarrow (a, \alpha)$
2 $v \leftarrow$ **lookup** *live_target*(k) *// late binding; TTL cache permitted*
3 **if** $v \neq \perp$ **then**
4 cache$(k) \leftarrow v$; **return** definition(v)
5 **else if** cache$(k) \neq \perp$ **then**
6 **return** definition(cache(k)) *// degrade, do not fail*
7 **else**
8 **raise** unresolved(k) *// explicit, never a silent default*

invariant: a client never names a version. Consequently a release changes behaviour with no client deployment, and a rollback is observed by the next call rather than by the next deploy.

The read path. Algorithm 2 is deliberately eight lines long: the client names an alias, never a version, and a resolver that cannot reach the control plane serves the last known version rather than failing the call.

The shipped `caliber.resolver.PromptResolver` implements this deployment with a configurable TTL, an optional maximum stale age, payload-free telemetry, and explicit failure when no known-good value exists. The scope of that statement is worth fixing precisely, because "CALIBER is not on the serving path" is the kind of claim that is true of one deployment and false of another. It holds for the *resolver-library* deployment, in which the agent process links the resolver and calls the provider directly, and CALIBER's HTTP surface is contacted only to resolve an alias. It does *not* hold for deployments that route inference or tool execution through CALIBER's own endpoints — a published workflow served over HTTP (§7), or a tool invoked through the governed execution path — where CALIBER is squarely in the request path and its cost is not bounded by Algorithm 2. E1 in §10 measures the first case, and a reader sizing the second should read E1 as a lower bound rather than as the answer.

Execution lifecycle. Figure 8 traces one refinement end to end across seven components. The properties worth reading off it are structural rather than incidental: exactly two steps are human decisions; the enforced gate sits before the candidate reaches review, so an operator is never asked to adjudicate a candidate that failed its own evaluation; and the promote step reads the outgoing alias target and commits it in a durable release intent before rotation, so rollback and reconciliation use a recorded value rather than inferring one (Algorithm 4).

5.8 Extensibility

The layering pays off as a small number of places to add capability without touching the substrate: provider protocols for LLM, evaluation, and promotion, each one operation per pipeline stage, with a deterministic fake default so the server boots with no API key; a storage-backend protocol; four event transports; a tool-sandbox protocol; 31 workflow node types behind a typed IR shared by the in-server interpreter and the generated Agents SDK export; deterministic scorers

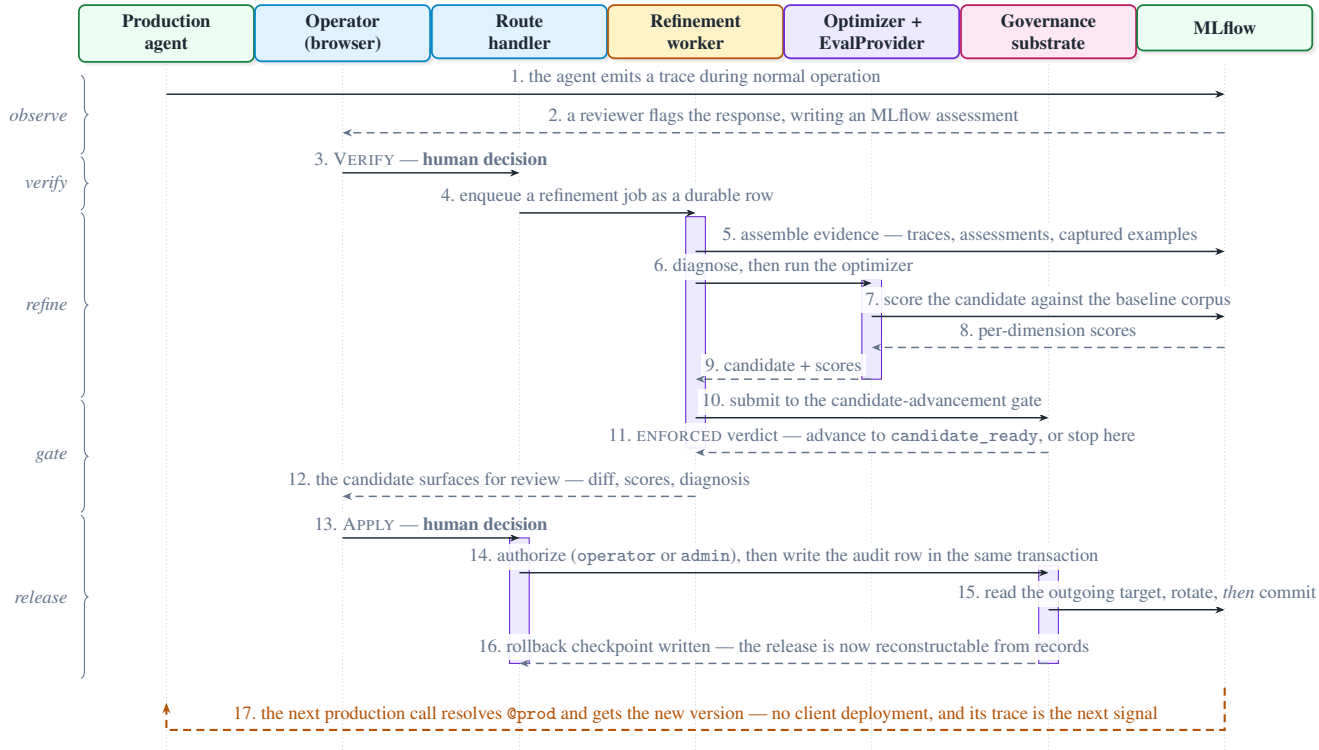


Figure 8: End-to-end sequence for one prompt refinement, from a flagged production trace to a rotated alias and the trace that closes the loop. Activation bars mark which component holds the work; the left-hand braces group messages into the phases of Figure 4. Three properties are meant to be read off this figure directly. First, exactly two steps are human decisions — step 3 confirms the failure is real and step 13 is an explicit operator action over the diff and the scores. Nothing between them requires a person, and nothing after step 13 asks for one. Second, the enforced gate at step 11 sits *before* the candidate reaches review, so an operator is never asked to adjudicate a candidate that failed its own evaluation. Third, step 15 reads the outgoing alias target and commits a release intent before rotation, so a crash leaves a durable reconciliation obligation with the exact value rollback must restore (Algorithm 4).

surfaces · control plane · governance · asset families · external systems · asynchronous · durable state

and authored judges referenced by token wherever a scorer is accepted; a peer-distributed plugin contract (`caliber-plugin-sdk`) that freezes extension-facing optimizer and provider shapes without pulling the server runtime into third-party packages; and two graph-extraction backends.

The deterministic fake default deserves a note, because it is a small decision with large consequences for the evaluation protocol in Appendix C: because every provider seam has a fake, the entire control plane is exercisable without a model provider, which is what makes the platform's own tests hermetic and its performance measurements separable from provider latency.

The Aria capability registry is the newest seam and the one we describe most cautiously. The intent is that an operation is declared once — key, risk tier, scopes, input schema, handler — and the agent toolset picks it up as a *projection* of the registry. The current state is 29 hand-written tools plus a 9-capability projection, so this is an emerging seam rather than achieved parity, and we count completing it as future work rather than as a contribution.

5.9 Scalability, reliability, and fault tolerance

Scalability. The request path is stateless and scales horizontally. Claim-based loops are safe at any replica count. The scaling limits are the relational store, which is the arbitration point for every queue; the per-process in-memory limiters described above; and — for the refinement path specifically — operator attention, since APPLY is a human decision by design (D4). We believe the last of these is the real limit in practice, and we say so rather than implying the system scales without a human bottleneck.

Reliability. Failure handling follows from where state lives. Workflow-run claims carry leases and resume from checkpoints; the refinement janitor marks expired-heartbeat jobs failed. Calibration deliberately has no automatic replay: an operator abandons an ambiguous running job or creates a new lineage-linked retry, because authored tools may have side effects. A crashed process mid-release may leave an external effect applied without its local record, which is the dual-write boundary above; those paths are idempotent so that retry converges. A rollback checkpoint is written on release, so recovery restores a recorded target rather than inferring one. Deployment aliases for workflows carry a checkpoint stack, and knowledge-base activation derives its prior state from activation history.

The failure domain the topology chooses. In the embedded topology, CALIBER and MLflow share a process: one crash takes both surfaces down, and this is appropriate for development. In the standalone topology they are separate failure domains connected over HTTP, so MLflow unavailability degrades CALIBER to the CALIBER-owned operations that do not need the registry. Prompt authoring, registry inspection, evaluation, and promotion are unavailable; local audit, workflow, tool, and recovery records remain available. The API and SPA are identical in both (Figure 9), which is what makes this a deployment decision rather than a product decision.

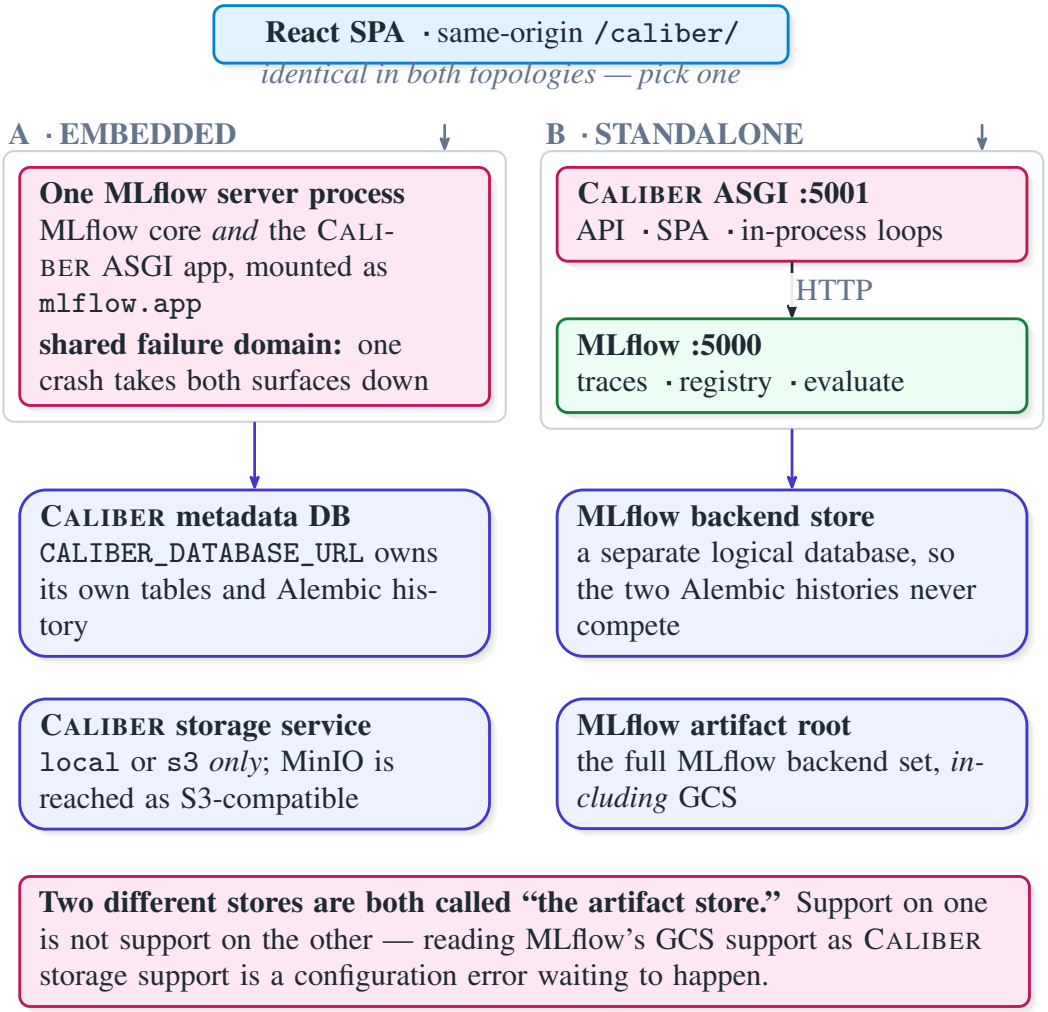


Figure 9: The two deployment topologies. One ASGI application, two ways to run it; the API and the SPA are identical, so the choice is a failure-domain and operations decision rather than a feature one. Topology A mounts CALIBER inside the MLflow server process as an `mlflow.app`, which is convenient for development and couples the two failure domains. Topology B runs CALIBER as its own service reaching a vanilla MLflow over HTTP. In neither mode is CALIBER a transparent gateway in front of MLflow: it is a sibling surface, not a proxy. Two configuration facts are called out because both are load-bearing and both are easy to get wrong. `CALIBER_DATABASE_URL` independently owns CALIBER's tables and is not required to equal MLflow's backend store — the bundled stack uses separate logical databases precisely so the two Alembic histories never compete for one version table. And "artifact store" names two different things: MLflow's artifact root supports its full backend set including GCS, while CALIBER's own storage service accepts `local` and `s3` only.

6 Core Components

This section describes the components whose behaviour the rest of the paper's claims depend on. We omit components that are conventional — configuration, persistence, the storage service — except where their contract is load-bearing.

6.1 The registries

Each family's registry owns typed definitions and their history, and the shapes differ because the families differ. Prompts hold immutable versions in the MLflow prompt registry behind aliases; CALIBER stores the control-plane metadata about them and does not duplicate the version bodies. Workflows hold editable drafts promoted to published version rows, with deployment aliases selecting one. Knowledge bases hold immutable build versions behind an active-version pointer. Skills hold a mutable current record plus immutable snapshots. Tools hold separate name-and-version rows with lifecycle status and no live alias. Test sets hold a version counter plus per-example validity intervals, which is what allows a score computed last month to be interpreted against the corpus as it stood then.

The uniformity is in the *interface* — list, read, version, diff — and not in the semantics behind it. Table 1 is the authority on the latter.

6.2 The evidence base

Evidence is the input to every measurement and the reason a release is reconstructable. Three sources feed it. Test sets are curated example collections with expectations, versioned so that a score is interpretable later. Production traces supply real inputs, and trace-to-example capture promotes a production failure into durable test data — which is the mechanism that stops the corpus from drifting away from what production actually does. Assessments carry human and automated judgements attached to traces.

A design point worth isolating: test sets can be synchronized to MLflow's native dataset registry for dataset-level lineage while the relational store remains authoritative, with a synchronization-state indicator surfaced in the UI. This is a concrete instance of the sibling-not-proxy relationship — CALIBER neither hides MLflow's dataset features nor pretends its own copy is a cache.

6.3 The evaluation framework

Evaluation composes deterministic scorers with authored LLM judges. Deterministic scorers are cheap, reproducible, and appropriate for structural properties — schema conformance, required-field presence, refusal detection, latency and cost budgets. Authored judges are created from natural-language instructions through MLflow's judge-authoring API, stored as governed artifacts, and referenced by a `Judge.<id>` token anywhere a scorer is accepted: in the refinement gate, in human review, and in a scorecard.

Because judges are themselves LLM calls, they inherit the biases documented for LLM-as-judge evaluation [39, 69]. CALIBER's response is structural rather than corrective: judges are governed artifacts with versions, and their agreement with human labels is measurable through review queues, so a judge's trustworthiness is an empirical quantity attached to the judge rather than an assumption made about it. Appendix C specifies how that agreement is to be measured; §12 is explicit that we have not yet measured it.

Human review is built CALIBER-native over the open-source assessment primitives: an operator defines a label schema — pass/fail, categorical, numeric, free text — enqueues traces, and reviews them, with answers written back onto each trace as assessments and expectations. This

keeps human labels in the same substrate as automated ones, which is what makes agreement computable at all. Completed binary answers can be imported directly into the judge-alignment surface. The import excludes incomplete and non-binary answers and retains the queue, item, trace, question, reviewer, completion, and assessment identifiers; this removes transcription as the default handoff without reclassifying an unreviewed item as a human label.

The workflow IR also carries two intentionally closed-vocabulary components for this evidence path. `data_transform` implements fixtures, field mapping, Draft 2020-12 JSON Schema validation, ordered decision tables, and weighted confidence without arbitrary code. `review_queue_enqueue` invokes the same audited, idempotent queue service as the UI and fails closed when a runtime enqueuer is unavailable. Arbitrary Python remains an extension boundary rather than the default representation of deterministic policy.

6.4 The calibration framework

Calibration is the machinery that proposes a measurably better version, and Figure 10 shows one job's internals. Five optimizer paths are implemented: two exposed on the prompt form, two selectable by policy, and one requiring explicit configuration. Non-prompt families use their own idioms — manifest replay for workflows, revision-fenced deterministic suites for tools, retrieval-quality calibration for knowledge bases.

Two properties of this component are architectural rather than algorithmic, and they are the ones that matter for the paper's argument. First, the diagnosis is persisted as a durable artifact rather than held in a prompt, so the reasoning behind a candidate outlives the job that produced it. Second, optimizer selection is a *policy* decision recorded on the job, so a later reader can tell which strategy produced a given candidate — without which a comparison across candidates is uninterpretable.

6.5 Gate semantics: the distinction the system turns on

CALIBER has two mechanisms that a reader may be tempted to call "the gate," and conflating them would invalidate several claims in this paper.

The candidate-advancement gate is enforced. It decides whether a refinement job may advance to a candidate-ready state. A failing gate stops the job. Nothing downstream sees the candidate; in particular, no operator is asked to review it. Within the refinement state machine this gate cannot be bypassed — but that scope is the whole claim. A direct operator release uses the durable release service but may carry an advisory verdict and attributed override without entering the machine.

The advisory release verdict is advisory. It is persisted per version as release evidence and surfaced in review. It does *not* block prompt alias rotation.

The deploy gate is family-specific. On asset paths that wire it — workflow deployments most notably — a deploy-gate policy enforces release under an optimistic alias check. It is not a cross-family policy.

The reason for placing the enforced gate on advancement rather than on release is the subject of D3 in Table 4, and is worth restating because it is counterintuitive. A gate that blocks release must be overridable, because verdicts go stale, corpora drift, and an urgent fix must be shippable at 3am. Once a gate is overridable, the override becomes the normal path, and the gate stops carrying information. Placing the unbypassable check earlier — where its only power is to withhold a candidate that failed its own evaluation — keeps it meaningful, because withholding a bad candidate is never the thing an operator needs to override.

Algorithm 3 sets the two procedures side by side, which makes the asymmetry structural rather than rhetorical. `advance` returns control flow: its `stopped` branch ends the job and no human is

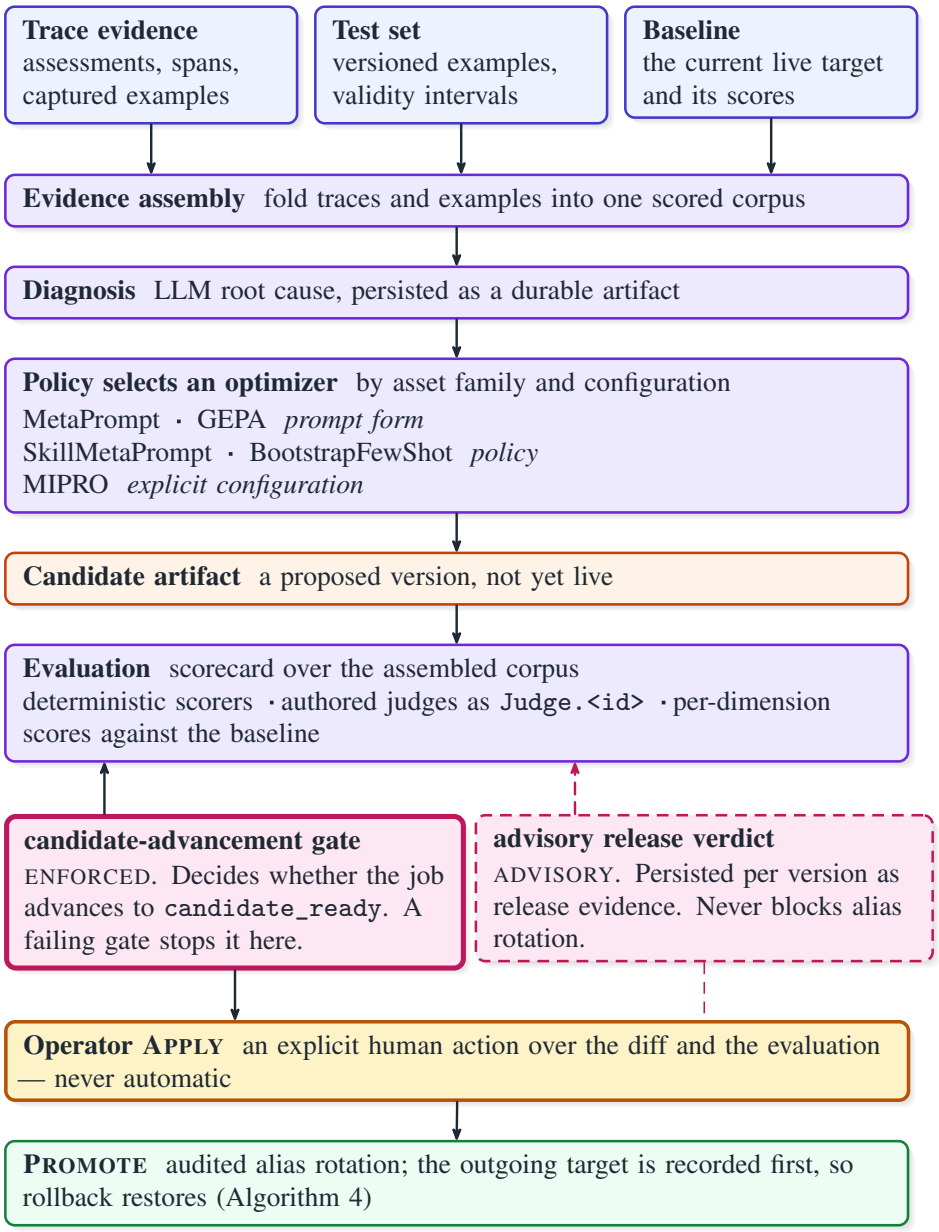


Figure 10: Inside one refinement job — the resolution at which the gate semantics become visible. Evidence assembly folds trace evidence and versioned examples into one corpus; diagnosis is persisted as an artifact rather than kept in a prompt; a policy selects among 5 implemented optimizer paths; and the resulting candidate is scored per dimension against the incumbent. The two gates are drawn differently because they are different mechanisms, and the paper depends on the distinction. The heavy-bordered candidate-advancement gate is *enforced*: it decides whether the job may advance to `candidate_ready`, and a failure stops the job. The dashed advisory release verdict is persisted per version as release evidence and never blocks alias rotation. Between them sits a human: APPLY is an explicit operator action over the diff and the scores, not a threshold that fires on its own.

Algorithm 3 The two gates, side by side. `advance` *stops the job*: its return value is control flow, and there is no override path because withholding a candidate that failed its own evaluation is never what an operator needs to override. Read "unbypassable" as scoped to this state machine: a direct authoring route can move a prompt alias without entering it (§12). `verdict` *files a record*: its return value is evidence, consumed by a human at APPLY. Placing the unbypassable check on advancement rather than on release is design decision D3 (Table 4).

procedure *advance*
// ENFORCED — the return value is control flow
in: v — a candidate; b, the incumbent baseline; C, the assembled corpus
out: `candidate_ready` ***or*** `stopped`

```
1  S ← ∅
2  for each scorer in scorecard(v) do
3      for each x in C do
4          S[scorer] ← S[scorer] ∪ {scorer(v, x)}
5
6  for each d in dimensions(S) do
7      if mean(S[d]) < mean(baseline(b, d)) − ε_d then
8          persist(v, S, stopped, reason = d)
9          return stopped                                  // no human ever sees v
10
11 persist(v, S, candidate_ready)
12 return candidate_ready
```

procedure *verdict*
// ADVISORY — the return value is evidence
in: v — a version, already released or releasable
out: *a persisted per-version record* — never a control-flow decision

```
1  R ← ⟨ scores(v), judges(v), corpus id, timestamp ⟩
2  persist_version_verdict(v, R)
3  // deliberately absent: any branch on R that could block
4  // rotate(). A stale verdict must never gate an urgent fix.
5  return R                                                // surfaced in review
```

ever shown the candidate. `verdict` returns a record: it persists evidence and, by construction, contains no branch that could block a rotation. The absence on lines 3–4 of that second procedure is the design, not an omission.

6.6 Release control

Release is where the audit obligations concentrate. Prompt creation and version authoring now register immutable drafts and cannot rotate an alias. Every prompt alias promotion and rollback enters the state machine in Algorithm 4: CALIBER commits the exact outgoing and incoming versions, actor, scopes, evidence, and an operation identifier before calling MLflow. It then marks the operation `applying`, performs an idempotent alias assignment, and settles it as `applied`; an exception becomes `reconcile_required`. The release API accepts the operation identifier on retries, and an operator-visible reconciler settles incomplete rows from the alias actually observed in MLflow.

This closes the unrecorded-effect window for prompt aliases, not for every family. Workflow deployments maintain a checkpoint stack that rollback pops; knowledge-base activation derives

Algorithm 4 Intent-first release for a prompt alias. The exact outgoing and incoming versions, actor, scopes, and evidence are committed before the provider call. A crash after the call therefore leaves an observable `applying` operation, not an unrecorded production change. Alias assignment to the same version is idempotent, so the operation identifier is also the retry key. This algorithm is implemented for prompt alias promotion and rollback; other families retain the weaker guarantees in Table 1.

procedure *release*

in: a — prompt asset; t, alias; v, immutable version; p, principal

in: o — caller-supplied or generated release operation identifier

out: *applied* — or a durable reconciliation obligation

if release[o] exists **then return** retry(release[o])
if `operator` $\notin$ scopes(p) **and** `admin` $\notin$ scopes(p) **then reject**
$u \leftarrow$ resolve(a, t) *// exact outgoing version*
begin transaction
  release[o] $\leftarrow \langle$`prepared`, a, t, u, v, p, scopes, evidence$\rangle$
  audit(`prepare`, o, a, t, u, v, p)
commit *// the recovery record now survives a crash*

begin transaction; release[o].status $\leftarrow$ `applying`; **commit**
try rotate($a, t \rightarrow v$) *// idempotent provider effect*
catch e
  release[o] $\leftarrow \langle$`reconcile_required`, $e\rangle$; **commit**; **return**
begin transaction
  release[o] $\leftarrow \langle$`applied`, provider result, *now*()$\rangle$
  audit(`promote`, o, a, t, u, v, p)
commit

procedure *reconcile*

in: o — an `applying` or `reconcile_required` operation

$x \leftarrow$ resolve(release[o].asset, release[o].target)
if $x =$ release[o].version_after **then** settle(o, `applied`)
else if $x =$ release[o].version_before **then** settle(o, `failed`)
else settle(o, `reconcile_required`) *// unexpected third state*

the prior active build from history; skill rollback restores a prior snapshot as a *new* current version, which is a different guarantee and is listed as such.

Adjacent to those family-specific live-target operations, the release-signoff domain records a candidate package rather than moving a target. A candidate contains weighted criteria, evidence references, blocking thresholds, a planned action, and a rollback target. The server recomputes its aggregate and blockers; an administrator may attach an attributable criterion-specific waiver, and the final go/no-go signoff freezes the candidate snapshot. A durable report job emits Allure-compatible criterion results plus a SHA-256 identity for that snapshot. This is evidence packaging and accountability, not distributed atomicity or proof that the referenced external evidence is available.

The audit guarantee, stated exactly. For a mutation whose effects are entirely database-resident and which calls the audit helper on the caller's session, the mutation and its audit row share one SQL transaction and cannot diverge. That is the whole claim. It does not extend to external MLflow or provider effects, which are dual-write boundaries; and it is a property of a call site, not of a module, so importing the helper proves nothing about coverage. We state it this narrowly

on purpose: an unqualified "everything is audited" claim would be unfalsifiable, and the qualified one is checkable.

6.7 The trust boundary

Figure 5 gives the admission and authority path, and Table 3 states the threat model row by row — adversary, what must be trusted, the asset, the boundary actually enforced, the residual risk, and where the claim can be checked. We put it in a table rather than in prose because prose let a reader assemble a stronger model than the one that holds: the evidence column makes it plain that every row is argued by construction and none is measured. Two entry classes have genuinely different admission rules: account clients present a session (or, in opt-in trusted-header proxy mode, a header) while published-service callers present either a public policy or a Bearer token, with a body cap, a quota, and idempotency handling. Both converge on one scope lattice in which `operator` and `approver` are sibling scopes each implying `viewer`, and `admin` implies all four.

Two properties need stating carefully, and both are weaker than a casual reading would suggest.

First, Aria capabilities are risk-tiered. Registry capabilities declared `gated` are withheld from the synchronous tool projection and require a plan interaction. The hand-written draft approval and publication tools are also classified `gated`; even a permissive build-mode session neither advertises nor dispatches them. The manual path uses the draft lifecycle UI/API. Two additional draft policies cross a separate service boundary: `agent_review` presents an immutable candidate hash, validation report, and test report to an isolated no-tools reviewer agent whose service identity must differ from the author and hold `approver`; `full_autonomy` continues an approved decision through a third, `operator`-scoped release identity. The decision records policy/model version, rationale, confidence, evidence identifiers, and expiry; invalid output, missing scope, shared identity, low confidence, expiry, or candidate drift fails closed. Goal-plan `gated` interactions remain human decisions. These gated paths are not yet one unified registry mechanism. The protection remains only as strong as future classifications, which is why the registry projection and its contract tests matter.

Second, local stdio MCP controls are *containment*, not a sandbox. Command and host allowlists, a sanitized environment, a private working directory, and process timeouts reduce ambient authority. Only an attested managed sidecar or a non-loopback HTTPS endpoint counts as an execution boundary for production preflight; the shipped bubblewrap profile is containment [16]. Similarly, fail-closed egress applies to governed HTTP and MCP transports and not to arbitrary user code: the shipped local tool runner has a separate interpreter, an empty environment, a private working directory, and resource limits, but retains ambient host filesystem and network authority. Untrusted authors require an operator-supplied OS-enforced backend [1]. Throughout this paper, "containment" and "isolation" are used as distinct terms, and the distinction is load-bearing rather than stylistic.

6.8 Published services and the copilot

A workflow may be published as an HTTP service under two-stage admission: a protected invoke validates its Bearer token before the body is consumed, and both public and protected invokes then count raw request chunks against a configured maximum before parsing, with policy and token revalidated under the enqueue lock. This is a per-request application bound and explicitly not connection-level flood protection — a distinction worth preserving, because conflating the two is how a body cap gets mistaken for a denial-of-service defence.

Table 3: Threat model. Each row names an adversary, what must be trusted for that row's boundary to mean anything, the asset it protects, the boundary actually enforced, what remains exposed, and where the claim can be checked. The evidence column is the honest one: every row is argued *by construction*, and **no row is supported by measurement** — §10 has no adversarial experiment, and a reader should not read this table as a security evaluation. Rows 1–3 are where CALIBER is weakest, and the containment-versus-isolation distinction they turn on is used consistently throughout (Figure 5).

	Adversary	Must be trusted	Protected asset	Boundary enforced	Residual risk	Evidence
1	Author of a local tool	the tool author	host filesystem, network, secrets	separate interpreter, empty environment, private working directory, resource limits	**containment, not isolation**: ambient host filesystem and network authority remain	by construction (§6)
2	Local `stdio` MCP server	the server binary	the same	command and host allowlists, sanitized environment, private cwd, process timeout	same as row 1; the shipped bubblewrap profile is containment [16]	by construction
3	Remote MCP endpoint	none	egress, tool inventory	fail-closed egress policy; production preflight demands an attested sidecar or non-loopback HTTPS	policy covers *governed* transports only, not arbitrary user code	by construction
4	Retrieved content (indirect prompt injection)	nothing	tool authority reachable from a run	capability risk tiers; `gated` capabilities withheld from the synchronous projection	classification errors; reviewer-agent quality remains unmeasured despite separate identities and hash binding	not measured [23]
5	Caller of a published service	none	enqueue capacity, workflow authority	Bearer validation before the body is read, body cap, quota, idempotency; policy revalidated under the enqueue lock	a per-request application bound, **not** connection-level flood protection	by construction
6	Account client escalating scope	the session store	live targets	one scope lattice; `operator` and `approver` sibling, `admin` implies all 4	no separation of duties: one operator may author and apply (§12)	by inspection
7	Holder of MLflow registry credentials	the registry's own access control	the prompt live target	none — outside CALIBER's control	an alias moved directly leaves no intent record and no evidence; CALIBER's guarantees are about *its* call sites	by construction (§12)

"By construction" means the boundary follows from the code path and is checkable by reading it; "by inspection" means it was verified at the call sites but is not enforced by a type or a test. Neither is a measurement. Closing row 1 and row 2 requires an OS-enforced backend [1, 16], which §13 records as the platform's sharpest outstanding security gap.

Aria, the embedded copilot, is a permissioned agentic tool loop with durable goal-plans and asynchronous resume: plans parked on long-running jobs are picked up by the plan worker when those jobs settle. It is a client of the same capability and authority model as every other surface, which is the property that keeps it from becoming a privilege-escalation path.

7 Capabilities

The previous two sections described structure. This one describes what the structure enables, organized by the requirements of §2.

7.1 R2 in practice: late-bound live targets

The single capability that most distinguishes CALIBER from a versioned prompt file is that client code resolves *which* version to run at call time. An agent loads `@prod`; the platform decides what `@prod` points to. A remediation therefore requires no client deployment, and the outer lane of Figure 2 exists to make exactly this visible.

The consequences are worth separating, because they are usually collapsed into a vague claim about agility:

- **Remediation latency decouples from deployment latency.** The bound on time-to-fix becomes the refinement path plus operator attention, not the CI pipeline. We specify how to measure this in Appendix C and have not measured it.
- **Rollback is a pointer move, not a redeploy.** Restoring the recorded target is a bounded database and registry operation rather than a code revert, and the target restored is the one the release *recorded* rather than one inferred as "the previous number." Rollback is itself a release operation, so it takes the same intent record and the same audit row as a promotion (Algorithm 4).
- **Authoring cannot move the pointer.** Registering a prompt version is a non-live operation: the shared registration helper rejects a request that would also set an alias, and directs the caller to the release service. That is what makes the two properties above hold for every in-repository call site rather than only for the path this paper describes.
- **Authority is scope-checked.** Rotating an alias requires the `operator` or `admin` scope. We are careful not to call this separation of duties: the sibling `approver` scope exists, but nothing currently requires that the principal who applies a change differ from the one who authored it, and the approval record is minted already-approved naming the acting operator. The Clark–Wilson property [11] is the target, not the present state (§12).

The honest caveat is that indirection is not free. A live target is a piece of mutable global state that determines production behaviour, so its integrity becomes critical. CALIBER's answer is that a rotation on the governed path is scope-checked, audited, and checkpointed — but that answer does not yet cover every route that can perform one, which makes the trade sharper than it first appears. §12 returns to it.

7.2 R3 in practice: evidence bound to versions

A score in a report tells you a number. A score attached to a version tells you what you knew when you shipped. CALIBER binds per-dimension scores, the corpus identity and version, the diagnosis, the selected optimizer, and the gate decision to the candidate, and mints a provenance anchor at release carrying that bundle.

The mechanism that keeps the corpus honest over time is trace-to-example capture: a verified production failure becomes durable test data, so the evidence base tracks what production actually does rather than what it did when the suite was written. This closes a specific drift problem that offline harnesses cannot close on their own, because they have no access to the production trace stream.

7.3 R4 in practice: one enforced gate, one human in refinement

§6 defined the gate taxonomy; here we state what it buys, and what it does not. The gate is enforced *within the refinement state machine*: a candidate that fails it never reaches review. It is not a system-wide interlock, because the direct operator promotion endpoint can carry an advisory verdict and an attributed override without entering that machine (§12). Authoring itself is non-live. Within the refinement path, the review queue contains only candidates that passed their own evaluation. An operator's attention is therefore spent on judgement rather than on triage — which matters, because operator attention is the binding constraint on this path (§5). Because the per-version verdict is advisory, an urgent release is never blocked by a stale artifact, and the override path that would otherwise erode the enforced gate never needs to exist.

7.4 Automation reach, stated as a boundary

Within the refinement path, CALIBER automates diagnosis, candidate generation, evaluation, gating, evidence assembly, and the mechanics of release. It does not automate the decision. The reach is therefore: from a flagged trace to a scored, gated, reviewable candidate without human involvement, and from a reviewed candidate to an audited release without human involvement. The human sits at exactly one point in between, and at one point before (verification).

Assistant drafts are outside that measured refinement chain. Their optional `agent_review` and `full_autonomy` policies replace the draft-review human with a separately scoped agent identity and, for release, a third service identity. They retain attributable records but have no measured reviewer-quality result; §12 therefore treats them as a scoped experimental exception rather than evidence against D4.

We want to be precise that this is a design position rather than an unfinished state. An auto-apply threshold would be straightforward to implement; D4 in Table 4 records why we consider it a mistake at the current state of the art, and §13 records the conditions under which we would revisit it.

7.5 Cross-family capabilities

Knowledge bases. Versioned retrieval corpora with chunking, embeddings, hybrid retrieval combining lexical and dense scoring [37, 55], cross-encoder reranking [47], and graph extraction over Apache AGE [6]. The governance-relevant property is that a build is an immutable version behind an active pointer, so "which corpus answered that question" is a resolvable query rather than an archaeological one.

Workflows. A visual editor over a typed IR with 31 node types, queued runs, runtime approvals, checkpointing, and publication as an HTTP service. The same IR backs the in-server interpreter and an Agents SDK export, which is what lets a workflow be governed here and executed elsewhere. The visual path includes closed-vocabulary deterministic transforms and direct review-queue enqueueing, so policy tables, schemas, confidence rules, fixtures, and review handoff do not require custom Python. A runtime-owned catalog exposes sixteen Cookbook examples; installing one creates a paused workflow and one editable draft in a single audited transaction, with prerequisites acknowledged before activation can be considered.

Tools and MCP servers. Tool definitions carry deterministic test suites fenced to a revision, so a suite result is attributable to a specific tool version. MCP servers are managed definitions with discovered tool inventories, audited edit history, fail-closed connection and policy controls, and a production preflight that distinguishes containment from an attested boundary (§6).

Gateway, signoff, and operator automation. The LLM Gateway is governed as a first-class integration surface rather than a side panel: endpoint discovery, guardrail attachment, per-model pricing, and trace-derived usage live beside the same evidence and audit substrate the release path uses. The release-signoff path adds typed candidates, weighted criteria, waivers, immutable go/no-go snapshots, and durable report jobs whose output is Allure-compatible JSON bound to a candidate hash. Those operational surfaces are exposed consistently across the SPA, the served management API, the remote SDK, and the CLI, so scripted operator automation does not depend on a browser-only flow.

Observability. MLflow tracing with multimodal attachments — extraction spans carry the source document, so a knowledge-graph or OCR pipeline's inputs are inspectable in the trace viewer — plus server-sent live events, opt-in archival of aged traces to object storage, and SLO reconciliation that produces durable incidents [9].

7.6 Capability coverage against comparable systems

Figure 11 places these capabilities against the systems a reader is most likely to be choosing between. It is a coverage grid rather than a scorecard: a cell records whether a capability exists as a first-class, wired concern, and says nothing about quality, performance, or maturity. Levels are encoded by fill *and* by glyph so the figure reads in greyscale.

The two rows where CALIBER is behind are included deliberately and are not incidental. CALIBER is not a multi-agent orchestration framework and does not intend to become one. It has no managed cloud offering, which for many teams is the deciding factor regardless of architecture. A coverage grid whose leftmost column won every row would be advertising rather than evidence, and a reader is entitled to treat such a grid with suspicion.

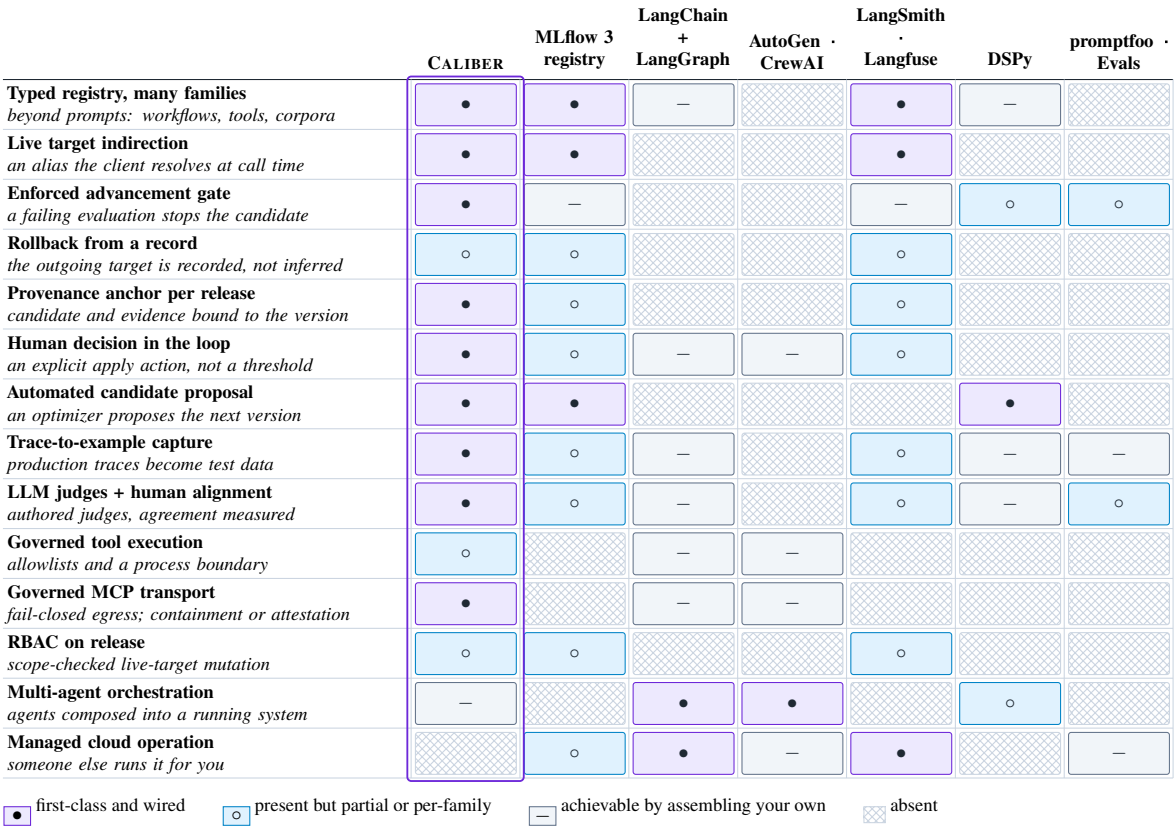


Figure 11: Capability coverage against comparable systems, **as of 2026-08-03**. This is a coverage grid, not a scorecard: a cell records whether a capability exists as a first-class, wired concern, and says nothing about quality, performance, or maturity. Levels are encoded by fill *and* by glyph so the figure reads in greyscale. Claims about the named products are sourced to dated documentation and compared row by row in Table 7 [34, 36, 45, 46]; these are live products and the grid will age. Note in particular that CALIBER shares — rather than leads — the pointer indirection, rollback, and candidate-proposal rows: an earlier version of this figure marked those against MLflow, LangSmith, and Langfuse in error, and the corrected grid gives CALIBER a much thinner lead. Where CALIBER is behind, the grid also says so: it is deliberately not a multi-agent orchestration framework, and it has no managed cloud offering, which for many teams is the deciding factor (§11, §12). A grid whose leftmost column won every row would be advertising rather than evidence.

8 Design Decisions

Table 4 records ten decisions, each with the alternative it displaced and the price it charges. This section expands the four that are most contestable, on the principle that a design record is only useful if a reader can disagree with a specific decision rather than with the system as a whole.

8.1 D1: reuse MLflow rather than owning the substrate

Building CALIBER's own trace store and prompt registry would have produced uniform semantics, one migration history, and one transaction domain — which would have eliminated the dual-write boundaries of Figure 7 outright. That is a real architectural benefit and we gave it up.

The reason is that the alternative competes with a mature system for no governance gain. MLflow already provides experiment tracking, trace storage, assessments, a prompt registry with alias semantics, an artifact store, a dataset registry, and evaluation entry points [44, 46]. Reimplementing that surface would have consumed the engineering budget the release path needed, and it would have forced any adopter who already runs MLflow to migrate trace history in order to gain governance. We have no data on how many teams that is, and the argument does not need any: the cost of reimplementation is paid whether or not the adopter already runs it.

The price is precisely stated in §6: the audit guarantee holds for database-resident mutations and not across the MLflow boundary. We consider a narrow checkable guarantee plus an explicitly named reconciliation boundary better than a broad guarantee that quietly does not hold, but a reader who weights uniform atomicity more heavily than adoption cost would reasonably choose differently.

8.2 D4: a human at refinement APPLY, always

This is the decision most likely to be contested, since the obvious research direction is to close the loop fully, and the machinery to do so already exists — the gate produces a decision and the promoter can act on it.

We keep the human for three reasons. First, a score improvement on an assembled corpus is evidence about that corpus, and the corpus is a sample assembled from *prior* failures; the failure modes that matter most in agent systems — tool misuse, over-confident assertion, injection reached through retrieved content [23] — are the ones such a corpus is least likely to represent. Second, judges are correlated with each other and with the model under evaluation [69], so a multi-dimensional pass is less independent evidence than its dimensionality suggests. Third, and most decisively for a control plane: an auto-apply threshold converts a visible, attributable change into an invisible one. The value CALIBER provides is that a change is reviewable; a change nobody reviewed is one nobody can be asked about.

This decision is scoped to the refinement path studied here. The assistant-draft subsystem now exposes an experimental agent-review path with distinct author, reviewer, and release identities plus hash-bound provenance (§12). That path supplies the mechanism for testing a non-human reviewer; without the calibration and off-corpus evidence in E3, it does not weaken the rationale for keeping refinement APPLY human.

The price is real and we do not minimize it: throughput is bounded by operator attention. If a fleet generates fifty candidates a day, CALIBER's design implies fifty reviews. §13 describes the two conditions under which we would revisit this — calibrated per-family confidence, and evidence that the judges generalize off-corpus — rather than treating the position as permanent.

Table 4: The load-bearing design decisions, each with the alternative it displaced and the price it charges. Recording the cost is the point: several of these trades are contestable, so a reader should be able to disagree with a specific row rather than with the system. §8 expands D1, D4, D5 and D9; §12 is largely the last column read back.

	Decision	Alternative	Why this one	What it costs
D1	Reuse MLflow for traces, prompt versions and evaluation	a self-contained store, with uniform semantics and one transaction domain	rebuilding tracking competes with a mature system for no governance gain, and would force a trace migration to adopt CALIBER	two stores with no shared transaction, so a reconciliation boundary is unavoidable (Figure 7)
D2	Per-family guarantees rather than one uniform contract	a single cross-artifact policy every family must satisfy	a test set and a prompt are not the same kind of thing; a uniform contract is vacuous or excludes useful families	the guarantee surface must be read per row and cannot be summarized (Table 1)
D3	Enforce the gate on *candidate advancement*; keep the release verdict advisory	one enforced gate blocking the release itself	a blocking gate must be overridable, the override becomes the normal path, and the gate stops carrying information	"gated" names two mechanisms and must be stated carefully every time (Algorithm 3)
D4	A human decision at refinement APPLY, always	auto-apply above a score threshold	score gains on a corpus assembled from past failures are necessary but not sufficient, and auto-apply makes a reviewable change invisible	refinement throughput is bounded by operator attention; assistant-draft agent review is a separate experimental path
D5	In-process loops with database-arbitrated queues; no worker tier	a broker with dedicated workers	claim and lease columns give durable arbitration with one deployable and no split-brain between broker and database	loop capacity is coupled to request capacity; in-memory limiters become per-process (Algorithm 1)
D6	Synchronous ORM behind predominantly async routes	async SQLAlchemy throughout	the workload is transaction-bound, not connection-bound, and the sync ORM is the more mature surface	every new blocking call site is a latent event-loop stall
D7	Relational metadata authoritative for the file *inventory*; object storage for *bytes*	treating the bucket as the inventory	a bucket listing cannot express versions, scopes or retention, so an inventory rebuilt from one is unauditable	orphaned bytes are possible after a failed dual write and need a sweeper
D8	Containment for local tool and MCP execution; attestation only for production	require OS-enforced isolation everywhere	requiring a sandbox runtime locally would make the common path unusable, and naming the weaker guarantee beats overclaiming	the shipped runner keeps ambient host authority, so untrusted authors need an operator-supplied backend (Figure 5)
D9	Ship a single environment in v1	a `dev` ⟶ `staging` ⟶ `prod` ladder	the ladder needs approval routing, requester/approver separation and per-stage config; a half-built one is a false assurance	no staging rehearsal, and a multi-environment mode in *this* path must still rebuild the pending-approval path (a separate Workspace subsystem has since built exactly that, on its own)
D10	Same-origin SPA served by the backend	a separately hosted frontend	one origin removes CORS and cross-site cookie handling from every governance route	frontend and backend release together; no independent frontend rollback

Each row is one decision, its displaced alternative, the reason, and the price. Alternating bands are a reading aid only.

8.3 D5: database-arbitrated queues, no worker tier

A broker with dedicated workers is the conventional choice and has real advantages: independent scaling of queue capacity, mature backpressure, and operational familiarity. We chose claim-and-lease columns on relational rows instead.

The argument is that a broker introduces a second source of truth about job state. When the broker says a job is in flight and the database says it is queued, the resulting split-brain is an operational failure that requires reconciliation tooling to diagnose. Claim columns make the database the single arbiter: a consumer claims a row atomically, holds a lease, and heartbeats; the janitor reaps expired leases. This is the transactional-outbox and lease pattern rather than a novel mechanism [30], and it yields one deployable with no split-brain.

The price appears in two places. Loop capacity is coupled to request capacity, because they share a process — a burst of queued work competes with request serving for the same resources. And every process runs its own loops, which is what makes the in-memory limiters per-process (§5.7). A deployment that needs to scale drain capacity independently of request capacity would be better served by a broker, and we would not argue otherwise.

8.4 D9: a single environment in v1

Shipping without a `dev` → `staging` → `prod` ladder is the decision most likely to be read as immaturity, so it is worth stating what the ladder actually requires: per-stage configuration and secrets, approval routing with a requester-and-approver distinction, promotion state that survives stage transitions, and compatibility tests across stages. CALIBER's current prompt record is a born-approved provenance anchor, which means a future multi-environment mode must *rebuild* a pending/approve/reject path rather than enable one. The workflow promotion state machine remains active for quality and graded-executor policy; only its human-approval policy ships disabled.

The relevant constants that mark these seams are visible in the codebase, and we note deliberately that flipping them is not a supported activation — a real activation needs configuration, migrations, authorization, and compatibility tests. Environment classification does still fail closed: production-class aliases require an attached passing deploy gate graded by a real configured executor by default.

We judged that shipping a half-built ladder would be worse than shipping none, because a staging environment that does not actually gate is a false assurance — the specific failure mode where a control plane becomes theatre. A reader who believes a partial ladder still beats none has a defensible position; we simply made the other call, and §13 treats the full ladder as the highest priority remaining work.

A separate, later initiative has since built environment-scoped promotion for a different product: CALIBER's multi-tenant "Workspace" subsystem implements four fixed environments with `promotion_order`, requester/reviewer approval routing through Change Requests, and per-environment release state, as its own module with its own service (`workspace_release_operations.py`) rather than a change to the path this paper describes. It does not resolve D9: the scope decision and its cost above are unchanged for CALIBER's single-tenant v1 product, and the two mechanisms share no code.

Table 5: The implementation, by the numbers. Every count here is regenerated from the source tree by `paper/scripts/gen_stats.py` at build time rather than transcribed, so it can be recomputed instead of trusted; the tree state is recorded as `a87457c6ea2-dirty`. Line counts are physical lines including comments and blanks, which is the cheapest reproducible definition rather than the most flattering one.

Component	Files	Lines
Python package (control plane)	416	170k
TypeScript / TSX (SPA)	327	202k
Test modules	410	189k
Interface and schema surface		
Route modules / HTTP declarations	63	568
ORM models / request-response schemas	112	345
Alembic revisions / workflow node types	113	31
Architectural constants		
Layers / lifecycle modes	6	6
Asset families / RBAC scopes	9	4
Background loops / optimizer paths	10	5

9 Implementation

9.1 Scale and composition

Table 5 gives the implementation by the numbers. Every count in that table — and every count in this paper's prose — is produced by a script that walks the source tree at build time (`paper/scripts/gen_stats.py`) and emits LaTeX macro definitions that the document consumes. Nothing is transcribed by hand, so a number cannot drift out of agreement with the artifact it describes, and a reader with the repository can recompute all of them. Line counts are physical lines including comments and blanks: the cheapest reproducible definition rather than the most flattering one.

The backend is Python over Starlette and SQLAlchemy with Alembic migrations; the frontend is a React single-page application built separately with Vite and served *through* the Python package from the same origin, so both deployment topologies present one origin to the browser (D10, Table 4). Peer packages at the repository root carry the remote Python SDK, the zero-runtime-dependency plugin SDK, and the non-interactive CLI; that separation is deliberate, so developer automation and third-party extensions do not need to install the server's full MLflow/SQLAlchemy/Starlette runtime to make an HTTP call or declare a plugin contract.

9.2 Three implementation choices worth recording

Synchronous ORM behind async routes. Route callables are predominantly async while the ORM is synchronous; selected blocking work is explicitly offloaded to a thread. The reasoning is D6 in Table 4: the workload is transaction-bound rather than connection-bound, and the synchronous ORM is the more mature surface. The cost is a standing review obligation — every new blocking call site is a latent event-loop stall — and we record it as a cost rather than presenting the choice as free.

A deterministic fake behind every provider seam. `LLMProvider`, `EvalProvider`, and `Promoter` each have a deterministic fake as the default, so the server boots and the full control plane is exercisable with no API key configured. Two consequences follow that are larger than the decision looks. The test suite is hermetic: no test requires a model provider, so 410 test modules run without network access or spend. And performance measurement is separable: control-plane latency can be measured independently of provider latency, which is what makes the protocol in Appendix C meaningful rather than a measurement of somebody else's inference service.

Migrations as an ownership boundary. 113 Alembic revisions manage CALIBER's schema, and CALIBER's database URL is configured independently of MLflow's backend store. The bundled development stack deliberately uses separate logical databases so that the two Alembic histories never compete for one version table (Figure 9). This is a small operational decision that prevents a specific and hard-to-diagnose failure — two migration tools disagreeing about one schema-version row.

9.3 Testing and quality gates

The suite spans three layers: backend tests under pytest with parallel execution and a separate integration marker for tests requiring external servers; frontend unit tests; and browser end-to-end tests. Static analysis runs a linter and a type checker over the package. All three layers can emit a common structured report format, and emitting results requires no JVM even though rendering the report does. The browser layer includes a separate Cookbook configuration that starts a migrated, isolated database and rejects adapters containing direct request, endpoint, database, or manifest-seeding shortcuts. Its executable journeys cover skill authoring and trigger tests, all-sixteen catalog visibility plus installation of every example as a separate paused draft, review-queue creation, and, added this session, the seeded two-week-alpha journey (`two-week-alpha-journey.spec.ts`, trace through diagnosis, candidate, evaluation, apply, and release/rollback) and the Workspace import-to-promotion journey (`workspace-journey.spec.ts`). The local CI mirror terminates the detached MLflow/uvicorn process group before deleting its state so a passing browser run does not leave a listener or database behind.

We are deliberately not reporting a test count or a coverage percentage as evidence of quality. Both are available, and neither supports the claims this paper makes; presenting them as though they did would be the kind of number-decoration §10 declines to engage in. What the suite does support is the hermeticity claim above: the control plane is exercisable end to end without a model provider, which is a checkable structural property rather than a quality score.

9.4 Deployment

Figure 9 gives both topologies. A bundled loopback-only container stack provides PostgreSQL 17 with pgvector and Apache AGE, S3-compatible object storage, MLflow, an AI gateway, and NATS, for development. It is explicitly not a production deployment template: it is loopback-bound, and it enables a convenience credential bootstrap that a network-reachable deployment must leave disabled. Saying so in the paper is not throat-clearing — a development stack mistaken for a production template is a realistic and severe failure mode.

10 Evaluation

10.1 What state this evaluation is in

We state this before anything else, because a reader is entitled to know it before reading a methodology. **The performance, scale, comparison, and human-subject measurements described in this section have not been executed for this draft.** A checked-in deterministic harness does execute eight controlled claim checks: conditional queue ownership, operator fencing of late calibration results, release intent ordering and reconciliation, prepared release abandonment, resolver outage fallback, and synchronous publication gating. Its machine-readable manifest labels them structural evidence and explicitly says they are not latency, throughput, replica-scale stress, or human-agreement evidence. Table 6 therefore still marks every quantitative cell TBM.

This is a real weakness of the paper and we are not going to characterize it as anything else. It is worth being explicit about why we chose an empty table over a populated one. Numbers that were projected, carried over from an earlier configuration, or produced under conditions the paper does not state are worse than absent numbers, because they are indistinguishable from measured ones to a reader and they propagate into citations. An empty cell is honest and can be filled; a plausible wrong number cannot be unfilled. §12 lists this as the paper's primary limitation.

What we do claim here is that the controlled interleavings pass in the recorded environment and disclosed worktree state, that the questions below are the right ones, that they are answerable, and that the protocol is specified precisely enough that the answers will be checkable rather than merely reported.

10.2 Evaluation questions and why these

Table 6 organizes the questions into five groups. The design principle behind the selection is that each question should be capable of *falsifying* a claim made earlier in the paper, rather than merely characterizing the system favourably.

E1 — Control-plane cost. §7 claims that late-bound live targets decouple remediation from deployment. That claim is only interesting if the indirection is cheap on the serving path. E1 measures the added latency of a governed read and of alias resolution, and the sustained throughput per replica. The falsifying outcome is a resolution cost large enough that teams would cache around it, which would defeat late binding.

E2 — Queue arbitration. §5.7 claims that the conditional claim gives concurrent mutual exclusion at any worker count, and that recovery after a crash is loop-specific rather than uniform. Both are testable, and both are the kind of claim that fails under concurrency rather than in a diagram. The falsifying outcome for E2(d) is any instance of two workers holding one row; for E2(f) it is any loop whose observed recovery differs from the row Table 2 assigns it. The deterministic harness exercises the conflicting-select interleaving and observes one winner. That is regression evidence for the conditional update, not the replica-scale experiment E2(d) asks for, so the table remains unmeasured.

E3 — The refinement path. This group contains the question we consider most important and most likely to be uncomfortable. E3(h) measures the enforced gate's agreement with expert judgement, and E3(i) asks whether authored judges generalize *off* the corpus they were calibrated on. §8 argues for keeping a human at APPLY partly because we expect off-corpus generalization to be weak; if E3(i) shows strong off-corpus agreement, that argument weakens and D4 should be revisited. We would regard that as a useful result rather than a refutation, but it is genuinely a result that cuts against a position this paper takes.

Table 6: Evaluation state. Each row names a question, the metric that answers it, and the measurement's current state. A *not measured* chip marks a quantity this draft has not measured; the protocol for each is specified in Appendix C in enough detail to be executed independently. No cell contains an estimate, a projection, or a figure carried over from an earlier configuration. Rows marked *structural* are properties established by construction or by inspection rather than by measurement, and those are reported as answered because their evidence is the implementation itself.

	Question	Metric	State
E1 — Control-plane cost			
a	Overhead added to a governed read on the serving path	p50 / p99 added latency	not measured
b	Cost of resolving a live target versus a hard-coded version	p99 resolution latency	not measured
c	Sustained request throughput per replica	req/s at fixed p99	not measured
E2 — Queue arbitration			
d	Do two workers ever own one row as replicas scale?	concurrent double-ownership	not measured
e	Drain throughput versus replica count	jobs/min	not measured
f	Does each loop's crash recovery match Table 2?	per-loop outcome and latency	not measured
E3 — The refinement path			
g	Wall-clock from verified signal to candidate-ready	end-to-end latency	not measured
h	Gate agreement with expert judgement on held-out candidates	Cohen's κ [15]	not measured
i	Do authored judges generalize off the calibration corpus?	off-corpus agreement	not measured
E4 — Release and recovery			
j	Time to roll back a released prompt	alias-restore latency	not measured
k	Is a release reconstructable from records alone?	audit replay success	not measured
E6 — Comparison against baselines			
o	Time and correctness of the same release task on each baseline	task time; records recovered	not measured
p	What does an adopter gain over MLflow aliases alone?	E4(k) elements recovered	not measured
E7 — Operators			
q	Do operators correctly state a family's rollback semantics?	correct answers / asset family	not measured
E5 — Structural properties			
l	Is the control plane exercisable with no model provider?	hermetic suite	yes
m	Does a DB-resident mutation share its audit transaction?	inspection	yes
n	Per-family guarantee surface	Table 1	stated

E4 — Release and recovery. §6 claims that rollback is a restore of a recorded target and that a release is reconstructable from records. E4(k) is the operationalization of the paper's central value claim: given only the durable records, can an independent party reconstruct what changed, on what evidence, and approved by whom? The falsifying outcome is any release for which the records are insufficient.

E6 — Comparison against baselines. A control plane measured only against itself establishes nothing about whether building it was justified. E6 runs the same release tasks on Git-plus-CI, on MLflow aliases alone, on LangSmith, and on Langfuse [34, 36, 46]. E6(p) is the one we consider most dangerous to our own position: it asks what an adopter gains over MLflow alone, and §11.1 has already conceded that MLflow supplies versioning, aliases, rollback, and optimization. If E4(k)'s four elements all prove recoverable from MLflow alone, the paper's central value claim is substantially weakened.

E7 — Operators. §5.1 argues that per-family guarantees are the honest factoring, and §5 names the risk that operators will read one version-history panel as one guarantee. Whether they do is a question about people, and E7 asks it directly rather than assuming the answer. A falsifying outcome makes the capability-interface design a requirement rather than future work.

E5 — Structural properties. Three properties are established by construction or inspection rather than by measurement, and are reported as answered because their evidence is the implementation. The control plane is exercisable with no model provider configured, because every provider seam has a deterministic fake (§9). A database-resident mutation that calls the audit helper on the caller's session shares that transaction, which is a property of the call site and is verifiable by inspection. And the per-family guarantee surface is stated in Table 1. We separate these from E1–E4 deliberately: they are verifiable, but they are not measurements, and presenting them in the same category would inflate the apparent empirical content of the paper.

10.3 Protocol summary

Appendix C gives the full specification. Its load-bearing features are these. Measurements separate control-plane cost from provider cost by running the deterministic fake providers, so E1 measures CALIBER rather than an inference service. Concurrency tests for E2 run $N \in \{1, 2, 4, 8\}$ replicas against one database with a fixed job population and assert on the absence of concurrent double ownership — a correctness property — rather than on a low duplication rate. Delivery semantics after a crash are measured and reported per loop, because the loops differ (Table 2). Gate-agreement measurements for E3 use a held-out candidate set labelled by annotators who did not author the judges, and report Cohen's κ with a confidence interval rather than a bare agreement percentage [15, 19]. Reconstruction tests for E4(k) are performed by a party given database and object-storage access but *not* given the original operator's account of what happened, which is the only way to test reconstructability rather than recall. Every E1, E4, and E6 measurement is run against four baselines on the same tasks, and Appendix C specifies a seven-cell fault-injection matrix whose expected observable is stated per cell — including one cell the paper predicts CALIBER will fail.

10.4 Threats to validity we can already name

Some threats do not require running the experiment to identify, and naming them now constrains how the eventual results should be read.

Single-deployment measurements do not establish scaling behaviour. E1 and E2 will be run on one hardware configuration and one database. Latency and throughput figures from such a run characterize that configuration; they are not a scaling model, and we will not present them as one.

Gate agreement is corpus-relative by construction. E3(h) measures agreement on a held-out set drawn from the same distribution as the calibration corpus. A high κ there says the gate is consistent with expert judgement *on that distribution*, which is exactly the limitation E3(i) is designed to probe. Reporting E3(h) without E3(i) would overstate what the gate establishes.

Reconstructability tests measure sufficiency, not usability. E4(k) asks whether records are sufficient to reconstruct a release. It does not measure how long reconstruction takes or how much expertise it needs, both of which matter operationally and neither of which this protocol captures.

Baselines 2–4 cannot run most of the task set. CALIBER's non-prompt families have no counterpart in MLflow, LangSmith, or Langfuse, so E6 can only report scope for those tasks rather than a comparison. That is a weaker claim than a timing win and we will not present it as a stronger one; it also means the head-to-head portion of E6 is confined to prompts, which is exactly the territory where CALIBER's advantage is thinnest.

The system has no adversarial evaluation. Nothing in E1–E5 probes prompt injection reached through retrieved content, malicious tool authorship, or a compromised MCP server [23]. CALIBER's governance model constrains the blast radius of these by bounding what a capability may do, but that containment is unmeasured and, per §6, is containment rather than isolation. A reader should not read E1–E5, once populated, as security evidence.

11 Comparison with Existing Frameworks

This section makes two comparisons, and separating them matters. The first is narrow and is where the paper is most exposed: against the platforms that already manage a prompt lifecycle. The second is broad and is about architectural shape. Doing only the second would let a shape argument stand in for a feature argument, which is how a comparison section becomes advertising.

11.1 The prompt-lifecycle platforms already do much of this

An earlier version of this paper claimed the release concern was unoccupied. That was wrong, and Table 7 states it plainly. As of 2026-08-03: MLflow's Prompt Registry provides commit-based, immutable prompt versions with diffs and mutable *aliases* [46]; LangSmith provides commit history, custom commit tags, reserved `staging` and `production` tags resolved at pull time, documented rollback to a prior commit, and owners-only control over tagging [34]; and Langfuse provides versions, `production` and custom *labels* resolved client-side at fetch time, rollback by re-assigning a label, and *protected labels* restricted by role [36]. MLflow additionally ships prompt optimization — `optimize_prompts()` over GEPA and metaprompting, driven by a labelled dataset and scorers [45].

Four claims a reader might expect this paper to make are therefore unavailable to it. Late-bound pointer indirection for prompts is not novel; neither is rollback by re-pointing; neither is restricting who may promote; neither is generating a candidate automatically. Where an earlier draft treated these as CALIBER's contribution, they are table stakes, and the honest reading of the top block of Table 7 is that CALIBER re-implements a solved problem for prompts.

What survives. Three differences do survive, and they are narrower than the old framing but they are checkable. *First, family heterogeneity.* These platforms govern prompts, and MLflow also governs models; workflows, tool definitions, MCP server configurations, and retrieval corpora are not registry citizens with versions and pointers. CALIBER's contribution is the claim that one mode vocabulary can span nine families — together with the finding, which is the paper's actual intellectual content, that spanning them costs uniform guarantees (§5.1). *Second, evidence as a precondition rather than an annotation.* All three platforms link evaluations to prompts; none of their documentation describes a mechanism that makes a promotion *fail* because a score fell. In CALIBER the advancement gate is such a mechanism — inside the refinement state machine, and, as §12 concedes, only there. *Third, the wiring rather than the parts.* MLflow optimizes prompts and MLflow versions prompts, but the path from a flagged production trace to an optimizer run to a gated candidate to a reviewed release is assembled by the adopter. CALIBER's claim is that this path is worth being a first-class architectural object.

That is a smaller claim than "the concern is unoccupied," and we would rather state the smaller one accurately. A reviewer who finds even this delta insufficient is making a reasonable judgement about significance rather than catching an error.

11.2 Why a ranking across the broader field would be the wrong output

The four groups in Table 8 are not substitutes. A team does not choose between LangGraph and CALIBER any more than it chooses between a web framework and a deployment pipeline; the realistic configuration is an agent composed in a framework, traced by an observability platform, optimized by a program optimizer, and governed by a control plane. Producing a single ranking across them would require a common objective they do not share.

What can be compared is the *organizing abstraction*, because that choice determines what each system can and cannot express. Tracking and observability platforms organize around the

Table 7: Feature-by-feature comparison with the prompt-lifecycle platforms — the closest systems to CALIBER, and the ones its contribution should be judged against. **Claims are as of 2026-08-03**, sourced to the cited documentation; these are live products and a reader checking later should expect drift. Above the rule are the four concerns an earlier draft of this paper got wrong: versioning, call-time pointer resolution, rollback, and role-restricted promotion are *present* in all three, and CALIBER is not novel in any of them. Below the rule is where a difference survives scrutiny. §11.1 argues both halves.

Concern	MLflow Prompt Registry	LangSmith	Langfuse	CALIBER
Prompt versioning	immutable commit-based versions, with diffs [46]	commit history per prompt [34]	version identifiers per prompt [36]	versions per governed asset — the same mechanism, not a better one
Call-time pointer	mutable *aliases* [46]	commit tags; reserved `production`, pulled by name at runtime [34]	*labels*; an unlabelled fetch serves `production` [36]	live target per family — **not a differentiator**
Rollback	re-point the alias [46]	rollback to a prior commit [34]	re-label an older version [36]	restore a *recorded* checkpoint — governed path only (§12)
Who may promote	registry permissions, where the deployment provides them	owners-only tag create and delete [34]	*protected labels*, by project role [36]	`operator` or `admin` scope — comparable, and no separation of duties (§12)
Families beyond prompts	models; no other artifact type is a registry citizen	datasets	datasets	nine families — workflows, tools, MCP servers, corpora, skills — under one mode vocabulary (Table 1)
Evidence bound to the version	evaluation integrated; the score annotates	experiments and datasets, linked	evaluations and scores, linked	scores, corpus identity, diagnosis, and optimizer choice bound to the candidate and carried into review
Score as a precondition on promotion	none documented: promotion is a pointer move	none documented	none documented	a failing gate stops the candidate — inside the refinement state machine, and only there
Automated candidate proposal	GEPA and metaprompting optimizers [45]	none documented	none documented	optimizers wired *to* the gate — the optimization is not the delta, the wiring is
Cross-family impact	none — no second family to impact	none	none	a change is relatable to dependents, because dependents are records

"None documented" means the mechanism was not found in the cited pages on the access date — a claim about the documentation, which a reader can check, rather than about the product, which we are not positioned to make. Tracing-only systems such as Phoenix [7] are excluded: they do not offer prompt lifecycle management and these rows would score them unfairly.

Table 8: Architectural comparison across the dimensions a systems reader asks about. These systems are not substitutes, so the table compares *shapes and trade-offs* rather than ranking them; §11 carries the reasoning, and the last row states what each design gives up, including CALIBER's. This table stops at shape: Table 7 compares the prompt-lifecycle column feature by feature against dated sources.

Dimension	CALIBER	Prompt lifecycle + obs. *MLflow, LangSmith, Langfuse; Phoenix (tracing only)*	Composition *LangChain, LangGraph, LlamaIndex*	Multi-agent *AutoGen, CrewAI*	Optimizers + harnesses *DSPy, promptfoo, Evals*
Organizing abstraction	the governed asset, across families	the trace, plus the versioned prompt	the chain or graph	the agent and its conversation	the program or test case
Architecture	layered control plane; one ASGI process	service plus SDK	library, inside the host app	library or agent runtime	compiler or CLI, out of band
Modularity	narrow protocols at named seams	exporters and hooks	very high: composition *is* the product	agent, tool, memory interfaces	optimizer and metric interfaces
Extensibility	new family, optimizer or scorer — each must be *wired*	scorers and dashboards	components, freely	agents and tools	optimizers and metrics
Memory & execution	relational metadata authoritative; claim-and-lease queues	append-mostly trace store	in-process, host-provided	conversation buffers	batch, outside serving
Scalability shape	stateless replicas; the DB is the limit	collector scales; the trace store dominates cost	whatever the host is	conversation length and token spend	the optimizer's evaluation budget
Developer experience	browser-first operate-and-release; big config surface	excellent for diagnosis	excellent for building	excellent for prototyping	excellent offline
Automation reach	trace → signal → candidate → gate → reviewed release	alerting; MLflow adds evaluate → optimize → alias, self-assembled	none, by design	task decomposition within a run	candidate generation only
Governance model	typed asset, live target, scopes, audit — per family	for prompts: version, alias, role-restricted promotion — but the score does not gate [36, 46]	none	ad hoc per conversation	none
What it gives up	**not** a composition or multi-agent framework; one environment; no managed cloud; per-family guarantees	governs prompts and models only; no score gates the promotion	no version, evidence or release semantics; every team rebuilds them	no governed release path; provenance is ad hoc	no live target, no audit trail, no human decision point

Rules set off the CALIBER *column; the tints and bands are reading aids and carry no other meaning.*

trace *and*, in the three cases discussed above, around a versioned prompt — so they are authoritative about what happened and, for prompts specifically, about what is live. What their organizing abstraction does not reach is a second artifact type: a system whose registry citizen is the prompt has no place to put a workflow definition. Composition frameworks organize around the chain or graph, which makes them excellent for construction and structurally unable to carry release semantics — the graph is constructed in the host process, so there is no artifact to version. Multi-agent frameworks organize around the agent and the conversation, which makes provenance per-conversation rather than per-artifact. Optimizers organize around the program or the test case, which yields candidates and leaves the release to the caller. CALIBER organizes around the governed asset, which is what lets one vocabulary carry version, evidence, authority, and a live target across families — and is also why it has nothing useful to say about how to compose an agent.

11.3 The dimensions where CALIBER is genuinely different

Memory and execution model. This is the sharpest architectural difference. The observability systems are append-mostly stores queried after the fact; the composition and multi-agent frameworks hold state in process for the duration of a run; the optimizers are batch processes outside the serving path. CALIBER is the only system in the table for which *relational metadata is authoritative* and for which durable queue arbitration is part of the architecture rather than an application concern. That is a consequence of being a control plane: a system that must answer "who approved this and when" cannot hold that state in a process.

Automation reach. Composition frameworks reach, by design, nowhere — they are libraries. Multi-agent frameworks reach into task decomposition inside a single run. Standalone optimizers reach to candidate generation and stop. The prompt-lifecycle platforms reach further than the previous paragraph's framing suggests: MLflow in particular spans evaluation, optimization, versioning, and alias promotion, so its reach ends at an *operator-assembled* release rather than at alerting [45, 46]. CALIBER reaches from a trace to a verified signal to a candidate to a scored gate to a reviewed release. The distinguishing property is not the length of that span but that it is a single wired path with a gate in it, rather than a set of capabilities an adopter connects — and §11.1 is where that distinction is argued rather than asserted.

Extensibility, with a caveat we owe the reader. The composition frameworks are more extensible than CALIBER in the ordinary sense: adding a component is trivial and requires no coordination. CALIBER's extensibility is narrower and has a specific qualification — adding a family requires *explicitly wiring* its modes and governance, per §5.1. A reader comparing extensibility columns should read CALIBER's as "extensible at named seams, with an obligation attached," not as "more extensible."

11.4 The dimensions where CALIBER is behind

Developer experience for construction. If the task is to build an agent, a composition framework is the better tool and CALIBER is not a substitute. CALIBER's developer experience is browser-first and oriented toward the operate-and-release loop; its configuration surface is large, which is a genuine onboarding cost that a library does not impose.

Multi-agent orchestration. AutoGen and CrewAI are purpose-built for composing agents into a running system, and CALIBER is not competitive there (Figure 11). Automatic optimization of a multi-agent *bundle* — as opposed to a single prompt or skill — is not a shipped CALIBER capability.

Managed operation. Every observability platform in the table offers a hosted option. CALIBER must be run by the adopting team, on their own PostgreSQL, object storage, and MLflow. For a small team this may outweigh every architectural consideration in this paper, and a comparison that omitted it would be dishonest.

Uniformity. An adopter who wants one release contract across all artifact types will find CALIBER's per-family guarantees unsatisfying. §5.1 argues this is the honest factoring rather than an incompleteness, but the argument does not eliminate the cost: understanding CALIBER requires reading Table 1 rather than one sentence, and that is a real burden on the adopter.

11.5 Positioning, stated plainly

CALIBER's claim is not that it does more than these systems, and it is not that the release path is unoccupied — for prompts, it is occupied, by at least three products, with mechanisms that closely resemble CALIBER's (§11.1). The claim is narrower on two axes. First, that the release path generalizes *beyond prompts* to workflows, tools, MCP servers, and corpora, and that a single mode vocabulary can span them. Second, that what the generalization costs — guarantees that are per-family rather than uniform — is a result worth reporting rather than a defect to be hidden.

Whether this warrants a separate control plane, or should be absorbed into a tracking platform, is a legitimate open question, and the evidence currently tilts toward absorption being feasible: MLflow already has the registry, the aliases, the evaluation harness, and the optimizer, and what it lacks is a second artifact type and a gate. An absorbing system would need to add a typed multi-family registry, an authority model that distinguishes authoring from promoting, and an enforced precondition on the pointer move. That is a smaller distance than an earlier draft of this paper implied.

12 Limitations

We group limitations by kind, because they demand different responses: an unrun experiment is a schedule problem, an unenforced boundary is a security problem, and a per-family guarantee surface is a design consequence.

12.1 Limitations of this paper

The quantitative evaluation has not been run. This remains the paper's primary limitation and it is not mitigated by anything else in it. §10 specifies the protocol; Table 6 contains no measurements. Claims about control-plane overhead, drain throughput, gate agreement, and rollback latency are therefore *unsupported* in this draft — not weakly supported. A reader should treat the architectural argument as the paper's contribution and the performance characterization as future work. Eight deterministic fault/interleaving checks are published separately as structural evidence; they do not change this boundary.

No user study. CALIBER's central value claim is that a release becomes reviewable. Whether operators actually review more effectively with it, or merely click through a richer interface, is an empirical question about people that this paper does not address and that no amount of systems measurement answers.

Single-system evidence. Every architectural claim is grounded in one implementation. We argue in §5.1 that per-family guarantees follow from the heterogeneity of the artifact types rather than from our particular engineering, but a single system cannot establish that a different factoring is impossible.

12.2 Limitations of the release path

These three were found by checking the paper's claims against the implementation rather than by reasoning about the design, and each is a case where an earlier draft of this paper claimed more than the code delivers.

The prompt fix is not a system-wide release theorem. Prompt creation and version authoring are now non-live, and all in-repository prompt-alias call sites use the intent-first service in Algorithm 4. This establishes a durable reconciliation obligation before a prompt alias changes. It does not prove that workflow, skill, knowledge-base, or future family effects have the same ordering; their guarantees remain those in Table 1. Nor does a repository call- site inventory prevent an external administrator from changing an MLflow alias outside CALIBER. The claim is therefore path- and family-scoped, not universal.

A durable intent is not distributed atomicity. The prompt provider call and SQL settlement still cannot commit atomically. A crash after alias assignment can leave an operation in `applying`; the difference is that its exact before and after versions are already durable and the operation is visible through the release-operations endpoint. Reconciliation observes the provider and settles `applied`, `failed`, or `reconcile_required`. This removes silent divergence but does not remove the need to operate the reconciler.

APPLY is an operator confirmation, not an independent approval. The route requires the `operator` scope and mints an approval record that is already in the approved state, with the acting operator recorded as the approver. There is a sibling `approver` scope, but nothing requires that the person who authored or requested a change differ from the person who applies it. This is a deliberate human decision point placed after an automated gate, and it is real; it is not separation of duties, and we no longer describe it as one.

Assistant-draft autonomy is a scoped exception, not a replacement for APPLY. The assistant now has structural separation for two draft policies: `agent_review` records an approver-scoped agent decision by an identity different from the author, and `full_autonomy` delegates publication to a third operator-scoped service identity. The decision is bound to the draft hash/version and rechecked at publication. This does not alter the refinement-job APPLY path above, gated goal-plan steps, or the paper's lack of measured reviewer calibration. The implementation evidence shows fail-closed mechanics, not that the reviewer agent makes production-quality decisions.

A signoff record does not validate its external evidence. The release candidate service recomputes criteria and blockers, controls waivers and final signoff, and binds its Allure-compatible report to a candidate snapshot. The report job currently completes in the application process; it is neither a distributed renderer nor evidence that an external trace, evaluation, or review reference is reachable in the target deployment. Operators must verify the exact release package and its referenced evidence before treating a ready status as a go.

12.3 Limitations of the architecture

Guarantees are per-family, and that is load-bearing. An adopter must read Table 1 rather than a summary sentence. We defend the factoring but not the cost: the most likely operator error in CALIBER is assuming a guarantee transfers across families because the same interface component is mounted in both places. The shared version panel is the canonical trap.

Two dual-write boundaries cannot be closed. Object-storage writes and external registry effects cannot join the caller's SQL transaction (Figure 7). Prompt-alias writes are intent-first and idempotently reconciled, which makes their divergence observable but does not make them atomic. Other provider and object-storage crossings retain family-specific compensation. A failed object write can still leave orphaned bytes requiring a sweeper.

In-memory limiters are per process. The failed-login throttle and the API rate limiter are per-process in-memory limiters, so their effective ceilings scale with the worker count (§5.7). An operator who sizes them as though they were global will provision a limit several times looser than intended. This is a genuine sharp edge and the documentation-only mitigation is unsatisfying.

Loop capacity is coupled to request capacity. Because there is no worker tier (D5), a burst of queued work competes with request serving inside the same process. A deployment needing to scale drain capacity independently would be better served by a broker.

A single environment. There is no `dev` → `staging` → `prod` ladder, so there is no staging rehearsal for a release. Because the current prompt record is a born-approved provenance anchor, a future multi-environment mode must rebuild a pending/approve/reject path with a requester-and-approver distinction rather than enable an existing one (D9). A separate, later, multi-tenant "Workspace" initiative has since implemented environment-scoped promotion with approval routing as its own distinct subsystem (§8); this limitation is unchanged for the single-tenant product described here.

12.4 Limitations of the security model

These deserve separate treatment because overclaiming here is more harmful than overclaiming about latency.

Containment is not isolation. Local stdio MCP execution and the shipped local tool runner reduce ambient authority — command and host allowlists, a sanitized environment, a private working directory, process timeouts, resource limits — but retain ambient host filesystem and network authority. They are not a security boundary against a malicious tool author. Untrusted authorship

requires an operator-supplied OS-enforced backend [1, 16]. Only an attested managed sidecar or a non-loopback HTTPS endpoint counts as an execution boundary for production preflight.

Risk tiering is only as good as the classification. Capabilities declared `gated` are correctly withheld from Aria's synchronous tool projection, including the hand-written approval and publication tools. The registry still does not cover every hand-written operation, so future classification drift remains a review and contract-test risk (§6).

Project scoping is not a verified isolation boundary. Visibility filters are applied per path. This is not the same as a repository-wide, verified multi-tenancy guarantee, and CALIBER should not be deployed as though it were one.

Audit coverage is a property of call sites, not modules. The transactional audit guarantee holds where the helper is called on the caller's session. Importing it into a module does not establish that every mutation in that module is audited, and we know of no static check that would.

Fail-closed egress does not cover arbitrary user code. It governs HTTP and MCP transports. A tool that makes its own network calls from inside the runner is constrained by the runner's environment, not by the egress allowlist.

Connector presets are configuration, not connectivity evidence. The incident, deployment-health, and service-health starters deliberately contain no credentials and use non-routable placeholders until an operator configures an endpoint and egress policy. Their paired fixtures establish workflow behaviour, not production-source fidelity or reachability.

The body cap is not flood protection. Published-service admission counts raw request chunks against a configured maximum, which is a per-request application bound. It is not connection-level or IP-level denial-of-service protection, and a deployment needing that must place it in front.

Provenance anchors are internal records. They are not cryptographic attestations in the in-toto or SLSA sense [48, 62]. They establish what CALIBER recorded, not what an independent verifier can prove.

Prompt injection is constrained, not solved. Bounding what a capability may do limits the blast radius of injection reached through retrieved or tool-returned content [23, 53]. It does not prevent the injection, and §10 notes that no adversarial evaluation has been run.

12.5 Limitations of scope

CALIBER is not a composition or multi-agent orchestration framework, has no managed cloud offering, and does not optimize multi-agent bundles. The Aria capability registry is an emerging seam — 29 hand-written tools plus a 9-capability projection — rather than achieved parity, and we count it as future work rather than as a contribution.

13 Future Work

We order this by what would most change the paper's claims rather than by implementation convenience.

Execute the evaluation. The protocol in Appendix C is the immediate obligation. E3(i) — whether authored judges generalize off their calibration corpus — is the measurement we most want, because a strong result there would undercut this paper's own argument for keeping a human at APPLY (D4). Designing an experiment whose outcome could weaken your position is the only way the position ends up meaning anything.

Multi-environment promotion. The highest-value architectural work, and the most involved. A `dev` → `staging` → `prod` ladder needs per-stage configuration and secrets, approval routing with a requester-and-approver distinction, promotion state that survives stage transitions, and cross-stage compatibility tests. Because the current prompt record is a born-approved provenance anchor, this is a rebuild of the approval path rather than the enabling of a flag (D9). The interesting design question is whether stages should be a first-class dimension of the live target or a separate axis — the former is simpler and the latter composes better with per-family release idioms. A separate, later, multi-tenant "Workspace" initiative has since built environment-scoped promotion with approval routing as its own distinct subsystem; it targets a different, multi-tenant product rather than the single-tenant path described here, so this item remains open work for *this* path, not resolved by it.

Capability interfaces over the asset families. §5.1 identifies the design CALIBER should have had: a thin base contract every governed asset satisfies, plus orthogonal capability interfaces — `Releasable`, `Rollbackable`, `EvidenceBearing` — that a family declares and is then *required* to implement. The payoff is not elegance. It converts Table 1 from documentation a reader must consult into a constraint a family cannot violate. A first machine-readable registry now emits the nine-family contract from `/capabilities` and tests every row's shape; it prevents UI and documentation from inventing missing fields, but it does not yet prove each route implements its declaration. The remaining work is a retrofit across 9 families and their routes, and it subsumes the registry projection described next.

Complete the capability registry. Approval and publication are now gated in the synchronous projection. Converging the remaining 29 hand-written tools onto the registry projection would make the seam real and, more importantly, would let risk-tier classification be checked mechanically. That directly addresses the classification gap in §12: a projection with mandatory tier declaration makes an unclassified mutation impossible to introduce, which is a stronger property than any amount of review.

Isolation as a first-class backend. The sandbox protocol already exists; what is missing is a shipped OS-enforced implementation. A microVM or seccomp-based backend [1, 16] would let CALIBER offer isolation rather than containment for untrusted tool authorship, which is currently the platform's sharpest security limitation.

Attestable provenance. Provenance anchors are internal records. Signing them and emitting in-toto-style attestations would make an artifact's history verifiable by a party that does not trust the CALIBER instance that produced it [48, 62]. This matters most for organizations that need to demonstrate governance to an external auditor, which is the population most likely to want a system like this.

Cross-family impact analysis. Today a candidate is evaluated against its own evidence. A prompt change, a knowledge-base rebuild, and a tool revision can interact — and no current mechanism scores the *bundle*. This is the architectural precondition for multi-agent bundle opti-

mization, and it is also the work most likely to require revisiting the per-family factoring, since a bundle gate would be the first genuinely cross-family enforcement point.

Global rate limiting. Moving the in-memory limiters to a shared store would remove the per-process ceiling described in §5.7. This is straightforward, and we list it because leaving a known sharp edge documented rather than fixed is a legitimate criticism of the current implementation.

Broader adversarial evaluation. Injection through retrieved and tool-returned content, malicious tool authorship, and compromised MCP servers all warrant measurement rather than the architectural argument §6 currently offers [23]. The interesting question is not whether CALIBER prevents injection — it does not — but how much the capability model actually bounds the blast radius, which is a quantity nobody has measured for any comparable system.

Automating APPLY, conditionally. We would revisit D4 only under two conditions, and we state them so the position is falsifiable rather than ideological: calibrated per-family confidence that accounts for corpus coverage, and a demonstrated off-corpus generalization result from E3(i). Absent both, an auto-apply threshold trades a reviewable change for an invisible one, which is the opposite of what this system is for. The shipped assistant-draft `agent_review/full_autonomy` mechanism supplies the identity, hash-binding, expiry, and fail-closed plumbing needed to run that experiment, but it does not satisfy either empirical condition and therefore does not automate the refinement-job APPLY path.

14 Conclusion

Production LLM-agent systems are instrumented for detection, and prompt platforms now provide real versioning, live labels, rollback, and permissions. Governance is still uneven across the workflows, tools, MCP configurations, skills, and retrieval corpora that determine agent behaviour. The remaining problem is therefore not an empty release ecosystem; it is a fragmented one with different guarantees by resource family.

CALIBER occupies that position. Its organizing abstraction is the governed asset: a typed record with version history, an authority model, an audit trail, and — where its family supports one — a live target that client code resolves at call time. Around it, six layers and six lifecycle modes compose into a governance chain carrying a flagged trace to a measured candidate, an enforced gate, an explicit human decision, and an audited release with a recorded rollback target.

The architectural result we would most want carried forward is a capability boundary. Adjacency in a layered stack confers capability *availability*, not *inheritance*: a family obtains lifecycle behaviour and governance enforcement only by explicitly wiring them. A meaningful base contract can still be uniform — identity, history, authority, and audit — while release, rollback, evaluation, and evidence remain declared capabilities. CALIBER declares that guarantee surface per family in a registry whose declarations are checked for mutual consistency and from which the guarantee table is generated — but it does not yet tie a declaration to the handler that discharges it, which is a limit of this implementation rather than an impossibility result. Two further results follow the same discipline: naming the two dual-write boundaries is what allows a precise, checkable audit guarantee to be stated at all, and distinguishing containment from isolation is what keeps a security claim falsifiable.

We are equally clear about what this paper does not establish. The quantitative evaluation is specified and unrun; Table 6 is empty by choice rather than by omission, because a plausible unmeasured number is more damaging than a visible gap. There is no user study, and the claim that governed releases are more *reviewable* is therefore argued rather than demonstrated. And the evidence is from one implementation, which cannot show that a different factoring is impossible.

What we believe the architecture does establish is narrower and, we hope, more durable: that the release path for non-code agent resources is a coherent first-class concern, that occupying it requires an organizing abstraction the adjacent systems do not have, and that being precise about where the resulting guarantees stop is what separates a control plane from a dashboard with opinions.

Availability

CALIBER is released under the Apache 2.0 license. The implementation identifier `a87457c6ea2-dirty` records the base revision used for every count in §9; the `-dirty` suffix explicitly means those counts include working-tree changes. The manuscript source, generated counts, and build instructions are under `paper/` in the working tree supplied for review.

The manuscript source is versioned in the same repository as the implementation it describes, so a reader can check the paper against one disclosed source state. In a dirty checkout, the base revision alone does not identify the uncommitted patch; the `-dirty` marker is a disclosure, not a reproducibility claim. What can be reproduced is the document and its source-derived counts: running `make repro` in `paper/` regenerates `generated/stats.tex`, fails if the result differs from the committed files, forces a four-pass build, and runs the document checks. It also re-runs the eight deterministic structural checks without rewriting their recorded manifest. `paper/REPRODUCE.md` records the required toolchain and states the boundary of that command.

What that does *not* establish is the unrun performance, scale, baseline, and human-study protocol, which needs a pinned experiment environment and remains the first obligation before Table 6 can contain quantitative results.

Bibliography

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.

[2] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Ion Stoica, Omar Khattab, and Matei Zaharia. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

[3] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019.

[4] Anthropic. Model context protocol specification. https://modelcontextprotocol.io, 2024. Accessed 2026-08-03.

[5] Apache Software Foundation. Apache Airflow. https://airflow.apache.org, 2025. Accessed 2026-08-03.

[6] Apache Software Foundation. Apache AGE: A graph extension for PostgreSQL. https://age.apache.org, 2025. Accessed 2026-08-03.

[7] Arize AI. Phoenix: AI observability and evaluation. https://github.com/Arize-ai/phoenix, 2025. Accessed 2026-08-03.

[8] Denis Baylor, Eric Breck, Heng-Tze Cheng, Noah Fiedel, Chuan Yu Foo, Zakaria Haque, Salem Haykal, Mustafa Ispir, Vihan Jain, Levent Koc, Chiu Yuen Koo, Lukasz Lew, Clemens Mewald, Akshay Naresh Modi, Neoklis Polyzotis, Sukriti Ramesh, Sudip Roy, Steven Euijong Whang, Martin Wicke, Jarek Wilkiewicz, Xin Zhang, and Martin Zinkevich. TFX: A TensorFlow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2017.

[9] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[10] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *IEEE International Conference on Big Data (BigData)*, 2017.

[11] David D. Clark and David R. Wilson. A comparison of commercial and military computer security policies. In *IEEE Symposium on Security and Privacy*, 1987.

[12] Cloud Native Computing Foundation. Open policy agent. https://www.openpolicyagent.org, 2025. Accessed 2026-08-03.

[13] Cloud Native Computing Foundation. OpenTelemetry specification. https://opentelemetry.io/docs/specs/otel/, 2025. Accessed 2026-08-03.

[14] Cloud Native Computing Foundation. OpenTelemetry semantic conventions for GenAI systems. https://opentelemetry.io/docs/specs/semconv/gen-ai/, 2025. Accessed 2026-08-03.

[15] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 1960.

[16] Containers Project. Bubblewrap: Unprivileged sandboxing tool. https://github.com/containers/bubblewrap, 2025. Accessed 2026-08-03.

[17] CrewAI, Inc. CrewAI: Framework for orchestrating role-playing autonomous agents. https://github.com/crewAIInc/crewAI, 2025. Accessed 2026-08-03.

[18] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024.

[19] Joseph L. Fleiss. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76 (5), 1971.

[20] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1987.

[21] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12), 2021.

[22] Jim Gray. The transaction concept: Virtues and limitations. *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, 1981.

[23] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[24] Guardrails AI. Guardrails: Adding guardrails to large language models. https://github.com/guardrails-ai/guardrails, 2025. Accessed 2026-08-03.

[25] Pat Helland. Life beyond distributed transactions: An apostate's opinion. In *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007.

[26] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[27] Iterative, Inc. DVC: Data version control. https://dvc.org, 2025. Accessed 2026-08-03.

[28] Andrew Kane. pgvector: Open-source vector similarity search for PostgreSQL. https://github.com/pgvector/pgvector, 2025. Accessed 2026-08-03.

[29] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

[30] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[31] Kubeflow Authors. Kubeflow pipelines. https://www.kubeflow.org/docs/components/pipelines/, 2025. Accessed 2026-08-03.

[32] LangChain, Inc. LangChain and LangGraph. https://github.com/langchain-ai/langchain, 2025. Accessed 2026-08-03.

[33] LangChain, Inc. LangSmith: Observability and evaluation for LLM applications. https://docs.smith.langchain.com, 2025. Accessed 2026-08-03.

[34] LangChain, Inc. Manage prompts in LangSmith. https://docs.langchain.com/langsmith/manage-prompts, 2026. Documents commit-based version history, custom commit tags, the reserved `staging` and `production` tags, `client.pull_prompt("name:production")` resolution, rollback to a previous commit, and owners-only tag permissions. Accessed 2026-08-03.

[35] Langfuse GmbH. Langfuse: Open source LLM engineering platform. https://github.com/langfuse/langfuse, 2025. Accessed 2026-08-03.

[36] Langfuse GmbH. Prompt version control. https://langfuse.com/docs/prompt-management/features/prompt-version-control, 2026. Documents version identifiers, the `production` and `latest` labels plus custom labels, client-side resolution by label at fetch time, rollback by re-assigning the `production` label, and protected labels restricted by role. Accessed 2026-08-03.

[37] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[38] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research (TMLR)*, 2023.

[39] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[40] LlamaIndex, Inc. LlamaIndex: Data framework for LLM applications. https://github.com/run-llama/llama_index, 2025. Accessed 2026-08-03.

[41] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[42] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 2020.

[43] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019.

[44] MLflow Project. MLflow for GenAI: Tracing, evaluation, and assessments. https://mlflow.org/docs/latest/genai/, 2026. The current-capability source for claims about MLflow traces, assessments, scorers, and evaluation entry points. The 2018 paper predates all of these and is cited only for MLflow's original contribution. Accessed 2026-08-03.

[45] MLflow Project. Optimize prompts with MLflow. https://mlflow.org/docs/latest/genai/prompt-registry/optimize-prompts/, 2026. Documents `mlflow.genai.optimize_prompts()` with the GEPA and metaprompting optimizers, driven by a labelled dataset and scorers. Accessed 2026-08-03.

[46] MLflow Project. MLflow prompt registry. https://mlflow.org/docs/latest/genai/prompt-registry/, 2026. Documents commit-based prompt versioning, mutable aliases, prompt- and version-level tags, commit messages, and immutable versions. Accessed 2026-08-03.

[47] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.

[48] Open Source Security Foundation. SLSA: Supply-chain levels for software artifacts. https://slsa.dev, 2025. Accessed 2026-08-03.

[49] OpenAI. OpenAI agents SDK. https://github.com/openai/openai-agents-python, 2025. Accessed 2026-08-03.

[50] OpenAI. OpenAI evals. https://github.com/openai/evals, 2025. Accessed 2026-08-03.

[51] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[52] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[53] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022.

[54] promptfoo. promptfoo: Test and evaluate LLM prompts. https://github.com/promptfoo/promptfoo, 2025. Accessed 2026-08-03.

[55] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

[56] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1975.

[57] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2), 1996.

[58] Sebastian Schelter, Joos-Hendrik Boese, Johannes Kirschnick, Thoralf Klein, and Stephan Seufert. Automatically tracking metadata and provenance of machine learning experiments. In *Machine Learning Systems Workshop at NeurIPS*, 2017.

[59] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[60] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[61] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[62] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bytes and bits. In *28th USENIX Security Symposium*, 2019.

[63] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[64] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024.

[65] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024.

[66] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[67] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

[68] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 2018.

[69] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[70] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations (ICLR)*, 2023.

A Asset Families in Detail

Table 1 gives the guarantee surface in tabular form. This appendix records the reasoning behind the three rows most likely to surprise a reader, since the table has room for the conclusion but not for the argument.

Why tools have no live alias. A tool is a callable with a signature, and its callers bind to a specific `(name, version)` pair. Introducing an alias would mean a caller's tool could change signature underneath it between calls — a strictly worse failure mode than an explicit version bump, because it converts a compile-time-visible incompatibility into a runtime one. Tools therefore expose read-only family history and no live-target indirection. The corollary is that tool remediation *does* require a caller change, and we accept that cost rather than introducing silent signature drift.

Why skill rollback creates a new version. A skill has a mutable current record plus immutable snapshots. Rolling back restores a prior snapshot *as a new current version* rather than moving a pointer backward. The reason is that the skill's identity is the current record, and rewinding it would erase the fact that the intervening version ever existed — which is exactly the history a governance system exists to preserve. The observable consequence is that a skill's version number is monotonic even under rollback, which differs from the prompt path's alias semantics and is listed separately in Table 1 for that reason.

Why test sets have version counters and per-example validity intervals. A test set is evidence, and evidence must be interpretable at the time a score was computed. A monolithic version counter alone would force a new version for every example edit, making the history unusably granular. Per-example validity intervals let the corpus evolve while keeping any historical score interpretable against the corpus as it stood: given a score and its timestamp, the exact example set that produced it is recoverable. This is the mechanism that makes Figure 4's durable-residue claim true for measurements specifically.

Why the agent family exists at all. An agent in CALIBER is not an executable — agents are composed elsewhere (§3). It is the anchor record that verification items, refinement jobs, and approvals attach to, which gives those records a stable owner and gives operators one lever (`enabled`) that the workers read to pause and resume activity for a whole agent. Modelling it as a family rather than as a foreign key keeps the authority model uniform: pausing an agent is a scope-checked, audited mutation like any other.

B Interface Surface and the Governed-Asset Contract

B.1 The shape every registry presents

568 route declarations across 63 modules is too many to enumerate, and enumerating them would not tell a reader anything structural. What is structural is that each family's registry presents the same shape, and that the *semantics* behind identical-looking operations differ per family exactly as Table 1 records.

```
GET    /{family}                 list, project-scoped
POST   /{family}                 create a typed definition
GET    /{family}/{id}            read the current record
PATCH  /{family}/{id}            mutate  -- scope-checked, audited
GET    /{family}/{id}/versions   history -- SEMANTICS VARY (Tab. 1)
GET    /{family}/{id}/diff       compare two versions

# Present only where the family implements the facet:
POST   /{family}/{id}/test       bounded run against fixtures
POST   /{family}/{id}/evaluate   scorecard over assembled evidence
POST   /{family}/{id}/calibrate  enqueue a refinement job
POST   /{family}/{id}/promote    rotate the live target -- operator/admin
POST   /{family}/{id}/rollback   restore the recorded prior target
```

Listing 1: The shape shared by every family's registry surface. Identical operation names; per-family semantics. The comments mark where Table 1 is the authority.

The lower block is the interesting one. Its endpoints exist per family only where that family wires the corresponding facet, which makes the interface surface itself a reflection of §5.1: availability is structural, enforcement is wired. A client cannot discover a promote endpoint on a test set, because there is none to discover.

The shared `GET /capabilities` response makes that absence machine-readable: its `artifact_families` object declares history, live-target, promotion, rollback, evidence, gate, and calibration facets for all nine families. A contract test requires every row and field. This is stronger than a UI convention but weaker than a type system that proves every route implements the declaration.

That machine-readable surface is now published in four aligned forms. The management API serves its own OpenAPI document; the repository ships a lightweight remote Python client SDK (`caliber-sdk`); a thin operator CLI (`caliberctl`) over the same HTTP contract; and a separate plugin SDK (`caliber-plugin-sdk`) for in-process extension points that are intentionally not exposed through the remote management surface. The important property for this paper is alignment rather than packaging: browser, OpenAPI, SDK, CLI, and external automation all consume the same governed-asset management surface instead of a private side channel, while extension authors get a versioned contract that does not drag in the server runtime. The caveat is the same as above: typed access is a stronger discovery surface than a UI convention, not a proof that every route family has an equally mature wrapper.

B.2 What a promote must record

The prompt-alias service records the following fields before its external effect (Algorithm 4). Authoring routes cannot rotate an alias; the direct promotion, rollback, assistant-publish, and refinement APPLY call sites use the shared service. This is a verified prompt-family contract, not a system-wide invariant for every resource family (§12).

1. **The outgoing target.** What the live target pointed to before the rotation. Without this, rollback is a guess.

2. **The incoming version identity.** Which version is becoming live, by immutable identifier rather than by label.
3. **The evidence reference.** A refinement Apply records its approval, candidate/evaluation snapshots, and gate source; a direct operator release records the advisory gate state and any attributed override. Missing evidence is represented as `none`, not silently omitted.
4. **The authorizing principal and scope.** Who acted, under which of the four scopes.
5. **An audit row**, written on the caller's session where the mutation is database-resident (§6).
6. **A release operation state**, progressing through `prepared`, `applying`, and `applied`, or remaining visible as `failed`/`reconcile_required`.

Operators act on incomplete crossings through explicit recovery surfaces: `GET /releases/operations`, global and background reconciliation for `applying`/`reconcile_required`, and exact retry or reasoned abandonment for a provably pre-effect `prepared` row. Tool calibration follows a different rule: a reasoned retry creates a new lineage-linked job because repeating authored tool code automatically would be unsafe.

B.3 Admission, in the order it happens

Order matters here, and stating it prevents a specific misreading of the body cap described in §6.

1. For a protected published-service invoke, the Bearer token is validated *before* the request body is consumed. An unauthenticated caller cannot make the server read a large body.
2. For both public and protected invokes, raw request chunks are then counted against the configured maximum during bounded parsing.
3. Policy and token are revalidated under the enqueue lock, so a policy change between admission and enqueue cannot be raced.

This is a per-request application bound. It is not connection-level or IP-level flood protection, and step 1 does not change that — it only ensures the bound is applied to authenticated callers only where a token is required.

C Measurement Protocol

This appendix specifies the protocol for the questions in Table 6 in enough detail to be executed by a party other than us. None of it has been executed for this draft (§10). Where a choice would materially affect the result, we state the choice and the reason, so that a different choice by a replicator is visible as a deviation rather than absorbed as noise.

C.1 Common setup

Provider configuration. All measurements run with the deterministic fake `LLMProvider`, `EvalProvider`, and `Promoter` unless a row explicitly requires a real provider. This is the protocol's most important decision: it makes E1 and E2 measure CALIBER rather than an inference service, and it makes results reproducible across time, since a hosted model's latency is not a stable quantity. Rows that need real providers (E3) say so.

Topology. Standalone (topology B, Figure 9), so that CALIBER and MLflow are separate failure domains and CALIBER's own cost is separable. Report the embedded topology separately if measured at all; do not average the two.

Store configuration. PostgreSQL 17 with pgvector and Apache AGE; S3-compatible object storage; MLflow 3.15 or later with its own logical database. Record the schema revision and the tree revision (`a87457c6ea2-dirty`) with every result.

Warm-up and reporting. Discard the first 60 seconds of every throughput or latency run. Report p50, p95, and p99 — never the mean alone, since the control plane's tail is the operationally relevant quantity. Report the number of trials and the interval, not a single figure.

C.2 E1 — Control-plane cost

E1(a), governed-read overhead. Compare two clients issuing the same logical operation: one resolving a governed asset through CALIBER, one reading a hard-coded version from local configuration. Both must exercise the same downstream work, so the difference isolates the control-plane path. Vary offered load until p99 degrades, and report added latency at 50% of the saturation point rather than at saturation — added latency measured at saturation is dominated by queueing and is not the number an adopter needs.

E1(b), alias resolution. Measure resolution alone, with the downstream call stubbed. Report cold (first resolution after process start) and warm separately; reporting only the warm figure would hide whatever cache-miss cost exists.

E1(c), throughput per replica. Fix a p99 target, increase offered load until the target is violated, and report the sustained rate below it. Report the target, not just the rate: a throughput figure without a latency bound is uninterpretable.

C.3 E2 — Queue arbitration

E2(d), concurrent mutual exclusion under replication. Enqueue a fixed population of J jobs whose handlers record their own invocation to a uniquely-constrained table. Run $N \in \{1, 2, 4, 8\}$ replicas against one database. **Assert zero concurrent double-ownership**, not a low rate: this is a correctness claim and a statistical result would not support it. Repeat with induced contention by starting all replicas simultaneously against a full queue.

Note what this does *not* test. Exclusion under contention is not exactly-once execution across crashes, and the two diverge by loop (Table 2). Run E2(d) separately per loop and report its own semantics: at-most-once for refinement and calibration, at-least-once for the workflow-run

worker, the knowledge-base worker, and webhook delivery. A single aggregate number across loops with different recovery policies would be meaningless.

E2(e), drain throughput versus replicas. Same setup, measuring completion rate. Expect sub-linear scaling and report where it flattens — the flattening point is the database arbitration limit and is the number that matters for capacity planning.

E2(f), recovery after a mid-claim kill. Claim a job, kill the process uncleanly (`SIGKILL`, so no lease is released), and record what happens. The expected outcome is loop-specific and the test must assert the right one:

- **WorkflowRun** — the run is requeued on lease expiry and resumes from its last checkpoint. Measure the interval and check it against the configured lease timeout plus one sweep period.
- **Refinement** — the job is marked `failed` by the janitor and does *not* resume. Assert that terminal state and measure time-to-reap. An earlier version of this protocol expected resumption here; that expectation was wrong about the implementation.
- **Calibration** — no lease exists and no automatic replay occurs, so the row remains `running` until an operator records a reasoned `abandon` or `retry`. Assert that the original becomes terminal, a retry is a distinct lineage-linked row with the same immutable inputs, and a late worker result cannot overwrite the operator disposition.

Separately, kill the process *after* an external effect has been issued but before its ledger entry settles, and assert the effect is surfaced as `indeterminate` rather than silently retried or silently dropped.

C.4 E3 — The refinement path

E3 requires real providers, and results are therefore provider-specific. Record the model identifier and the date for every run.

E3(g), signal-to-candidate latency. Measure wall-clock from a verified verification item to a candidate-ready job, broken down per stage (Figure 10). Report the breakdown, not only the total: a total dominated by optimizer time and a total dominated by evidence assembly have entirely different engineering implications.

E3(h), gate agreement with expert judgement. The sample size must come from a power analysis rather than from a round number: fix the smallest κ worth detecting, the expected prevalence of withheld candidates, and the desired power, and derive n from those. Stratify by asset family and by task, because a gate that agrees well on prompts and poorly on workflows is not described by one pooled figure. Seal the test set and select the release threshold *before* it is opened; a threshold chosen after looking is not a measurement.

Have at least three annotators who *did not author the judges* label each candidate independently, and adjudicate disagreements into an expert gold set rather than resolving them by majority alone. Report Cohen's κ against the adjudicated label and Fleiss's κ among annotators, both with confidence intervals that account for clustering within family and task [15, 19]. Report per-class sensitivity and specificity separately: the cost of a false advance and the cost of a false withhold are not symmetric, and a single agreement number hides which one the gate is making. Report inter-annotator agreement even — especially — if it is low, since a low value bounds what the gate could possibly agree with.

E3(i), off-corpus generalization. This is the measurement designed to be able to embarrass the paper's own position (D4, §8). Calibrate judges on corpus A. Evaluate agreement on corpus B drawn from a *different* task distribution — not a held-out split of A, which would measure

something else. Report the agreement drop from A to B. A small drop weakens the argument for a mandatory human at APPLY; report it either way.

C.5 E4 — Release and recovery

E4(j), rollback latency. Measure from the rollback request to the first production call observably resolving the restored target. Include the client-side resolution, since a rollback that has been recorded but not yet observed by clients is not complete from the operator's point of view.

E4(k), reconstructability. The protocol's design matters more than its mechanics. Perform $R \geq 20$ releases with varied paths — some rolled back, some superseded, some with failing gates that stopped a job. Then give an independent party read access to *every authoritative store*: the relational database, object storage, *and* MLflow — but nothing else: no access to the original operator, no chat logs, no tickets. Ask them to state, per release, what changed, what evidence justified it, who authorized it, and what the prior state was. Score each of the four elements as recoverable or not, and report per-element rather than as a single success rate — the interesting failure is one element being systematically unrecoverable. Any element unrecoverable for any release falsifies the reconstructability claim in §4, and should be reported as such rather than averaged away.

Including MLflow in that access set is a correction rather than a convenience. MLflow owns prompt versions, aliases, traces, and assessments (Figure 7), so a protocol that withheld it could not reconstruct a prompt release even in principle, and would have measured the access model rather than the records. The stronger variant of this experiment — and the one worth running second — withholds MLflow and asks whether CALIBER can emit a *self-contained evidence bundle* that suffices on its own. That is a design requirement this paper does not yet meet, and §13 records it as work rather than as a property.

C.6 E6 — Baselines

A control plane measured only against itself establishes nothing about whether it was worth building. Every E1, E4, and E6 measurement is therefore run against four baselines as well as CALIBER, on the same task set and by the same participants:

1. **Git plus CI.** Prompts in files, review by pull request, release by deployment. This is what most teams do and is the baseline CALIBER must beat to justify itself at all.
2. **MLflow alone.** Prompt Registry versions and aliases, with `optimize_prompts()` and evaluation run by hand [45, 46]. This is the sharpest baseline, because it has most of CALIBER's mechanisms and none of its wiring; the delta it isolates is precisely the wiring.
3. **LangSmith** [34] and **Langfuse** [36], using commit tags and labels respectively.

E6(o), task time and correctness. Give each participant the same three tasks — diagnose a reported failure, produce and release a fix, and roll it back — on each system, counterbalanced for order. Report time-to-complete and, more importantly, the number of E4(k) elements a third party can later recover. A baseline that is faster but leaves nothing recoverable is a different point on the trade-off, not a worse system, and should be reported as such.

E6(p), the MLflow-alone delta. Run E4(k) against baseline 2 in isolation. The prediction CALIBER's design makes is that *what changed*, *what the prior state was*, and *who moved the alias* are recoverable from MLflow alone, and that *what evidence justified it* is not, because the evaluation result is not bound to the alias move. If all four elements turn out to be recoverable, the paper's central value claim is substantially weakened and should be reported as such.

E6, applied to non-prompt families. Baselines 2–4 cannot run the workflow, tool, or MCP-server tasks at all, since those artifacts are not registry citizens in any of them. Report that as a *scope* result rather than as a score: the comparison is not that CALIBER performs the task better but that the baseline has no mechanism for it, which is a different and weaker kind of claim than a timing win.

C.7 E7 — Operators

§5.1 argues that per-family guarantees are the honest factoring, and §5 names the resulting risk: an operator who sees the same version-history panel in five places infers five equivalent rollback guarantees and gets five different ones. That is an empirical claim about people and cannot be settled by a table.

E7(q), guarantee comprehension. Recruit ≥ 12 participants with production LLM-operations experience and no prior exposure to CALIBER. After a fixed onboarding, show each a version-history panel for each of five families and ask what rolling back would do. Score against Table 1. Report per-family accuracy, not a pooled figure: the interesting result is *which* family is misread, and the prediction is that tools — which have no rollback at all — are misread most often. Report confidence intervals and the onboarding materials verbatim, since comprehension is trivially inflated by better training.

A falsifying outcome here is not that CALIBER is slow but that its central design position is unusable: if operators cannot state the guarantee, documenting it per-family is not a mitigation, and the capability-interface design of §5.1 becomes a requirement rather than future work.

C.8 Fault injection

E2(f) covers one fault. The following matrix is the minimum for a paper that claims a recovery story, and each cell states the expected observable so a run either matches it or does not:

- **Worker `SIGKILL`** — per-loop outcome as in E2(f).
- **Database failover mid-transaction** — either no intent commits and no provider call begins, or a durable operation names the exact requested effect. Inject failure before and after each state commit.
- **MLflow unreachable during a release** — the rotation fails and the operation remains `reconcile_required`; reconciliation observes the prior target and settles it `failed`.
- **MLflow unreachable after a rotation but before the commit** — the operation remains `applying` or `reconcile_required`; reconciliation observes the requested target and settles it `applied`.
- **Object storage unavailable** — evidence write fails closed; a release does not proceed on missing evidence.
- **Clock skew across replicas** — lease expiry does not permit two owners. Skew each replica by more than one lease period.
- **Duplicate webhook delivery** — the receiver's settlement path absorbs it; report the observed duplicate rate rather than asserting zero.

Report each cell as pass, fail, or not run. A blank cell is more informative than a cell filled by argument.

C.9 Deviations to report

A replicator changing any of the following should say so, because each materially affects comparability: the provider configuration (fake versus real); the topology; the annotator pool's inde-

pendence from judge authorship in E3(h); the distributional distance between corpora A and B in E3(i); and whether E4(k)'s independent party had any channel to the original operators; the baseline versions used in E6, since these are live products; and the onboarding materials used in E7, since comprehension results are not comparable across different training.

D Operational Configuration Notes

CALIBER is environment-configured, and the surface is large — which §11 lists as a genuine onboarding cost. This appendix records the settings whose *semantics* a reader of this paper needs, rather than a full inventory.

The two database URLs are independent, deliberately. CALIBER's database URL is configured separately from MLflow's backend store and is not required to equal it. The bundled stack uses separate logical databases so the two Alembic histories never compete for one schema-version row (§9). Pointing both at one database is possible and is a mistake that surfaces later as a migration conflict rather than immediately as an error.

"Artifact store" names two different things. MLflow's artifact root supports MLflow's full backend set, including GCS. CALIBER's own storage service accepts `local` and `s3` only, with MinIO reached through S3 compatibility; anything else is rejected at construction. Reading support on one as support on the other is a configuration error waiting to happen (Figure 9).

Background work is gated at two levels. A master switch enables the loop lifecycle; four loops — the workflow-run, knowledge-build, and workspace-import workers and the scheduler — additionally have independent enable flags; the other six do not. Disabling the master switch leaves a deployment that serves requests and accumulates queued work indefinitely — which is the correct behaviour for a read-only replica and a silent failure for anything else.

Rate limits are per process. The failed-login throttle and the API rate limiter are in-memory, per-process limiters — a sliding-window counter and a token bucket respectively (§5.7). With MLflow's default of four gunicorn workers, a configured limit of L yields an effective ceiling near $4L$. Size the configured value as L/W for W workers, or pin a single worker.

The service body cap is per request. Published-service invokes count raw request chunks against a configured maximum, defaulting to 1 MiB. Callers needing to pass large content should place it in managed storage and pass a reference. Raising the cap raises a per-request bound and provides no flood protection (Appendix B).

Provider selection defaults to fakes. The LLM, evaluation, and promoter provider selections each default to a deterministic fake, so a fresh deployment boots and is fully navigable with no API key. The corollary matters for anyone interpreting a running instance: a deployment that appears to be evaluating candidates may be running the fake evaluator, and the provider selection should be checked before any score is believed.

The credential bootstrap is loopback-only. A convenience default credential is created only while the account table is empty, only under the loopback launcher, and never resets an existing password. Any network-reachable deployment must leave the insecure-default flag disabled and supply a strong password source. The bundled container stack is a loopback-bound development environment and not a production template (§9).

Single-environment constants are seams, not switches. The constants that mark the single-environment boundary are visible in both the frontend and the backend (D9, Table 4). Changing them alone is not a supported activation: a real multi-environment mode needs configuration, migrations, authorization, and compatibility tests (§13).